МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 4

Тема занятия: проектирование пользовательского интерфейса для ɾpa-робота
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: формирование у обучающихся представления о пользовательском интерфейсе RPA-робота и освоение навыков проектирования пользовательских форм в среде Puzzle RPA.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В ходе выполнения практической работы обучающиеся изучают назначение пользовательских форм в RPA-проектах, знакомятся с редактором форм Puzzle RPA, осваивают принципы выбора элементов интерфейса и их настройки, а также учатся проектировать форму ввода данных для автоматизированного процесса на этапе анализа и подготовки сценария роботизации.

Практическая работа направлена на развитие понимания того, что RPA-робот может взаимодействовать с пользователем не только через уведомления, но и через специально разработанный интерфейс, позволяющий задавать параметры работы робота до его запуска.

По умолчанию RPA-робот может работать полностью автоматически, без участия человека.

Но на практике часто возникает ситуация, когда роботу нужны данные от пользователя:

- выбрать файл для обработки
- указать дату, период или сумму
- ввести логин, комментарий, номер заказа
- подтвердить запуск сценария
- вывести результат работы робота в понятном виде

Для этого в Puzzle RPA используется пользовательский интерфейс – окна и формы, с которыми взаимодействует человек во время выполнения сценария.

Что такое пользовательская форма в Puzzle RPA

Пользовательская форма – это окно, которое робот показывает пользователю и которое содержит элементы управления:

- текстовые поля
- поля для ввода чисел
- списки выбора
- кнопки
- сообщения и подсказки

Пользователь вводит данные → данные сохраняются в переменные → робот использует их дальше в сценарии.

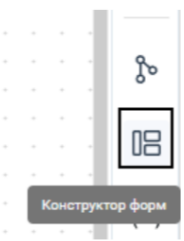
Важно: Форма не просто показывает информацию, она является источником входных данных для сценария.

Основные блоки, которые используются на практике:

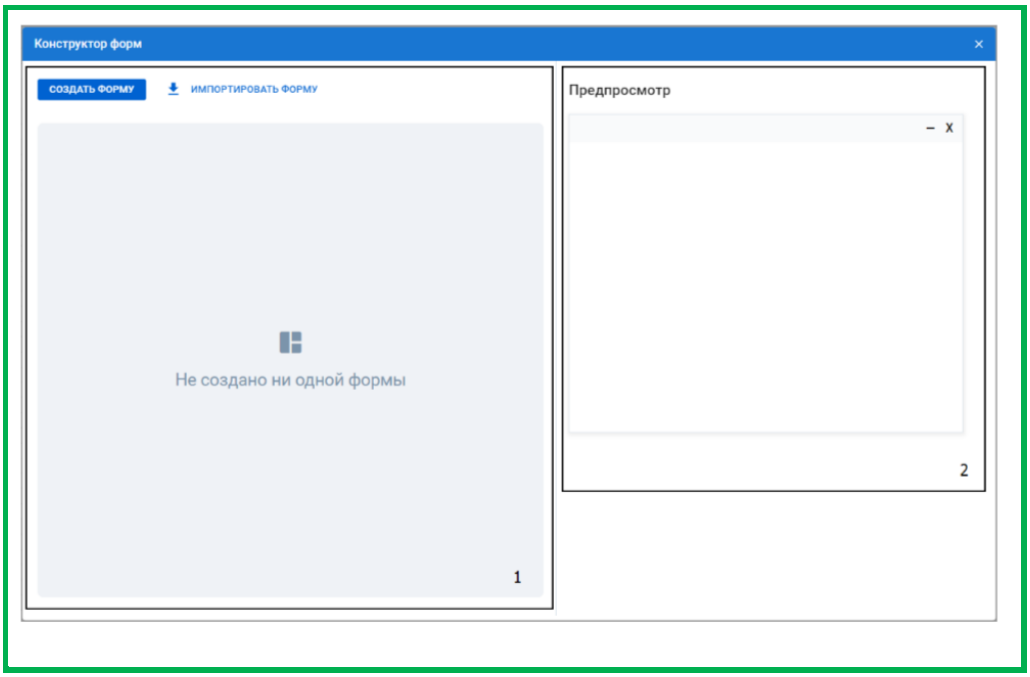
- Показать сообщение
- Ввести значение
- Пользовательская форма
- Ввод строки
- Ввод числа
- Выбор из списка
- Кнопка подтверждения

Конструктор форм — встроенный инструмент Puzzle RPA для визуального создания пользовательских интерфейсов ввода данных. С его помощью можно размещать на форме элементы управления — такие как текстовые поля, выпадающие списки, флажки, кнопки и настраивать их свойства. Созданные формы можно использовать для сбора параметров перед запуском алгоритма, настройки процессов или взаимодействия с пользователем в ходе выполнения сценария.

Чтобы открыть панель управления **Конструктор форм**, требуется кликнуть по кнопке, расположенной в Области кнопок взаимодействия с процессом:



Стартовый экран:



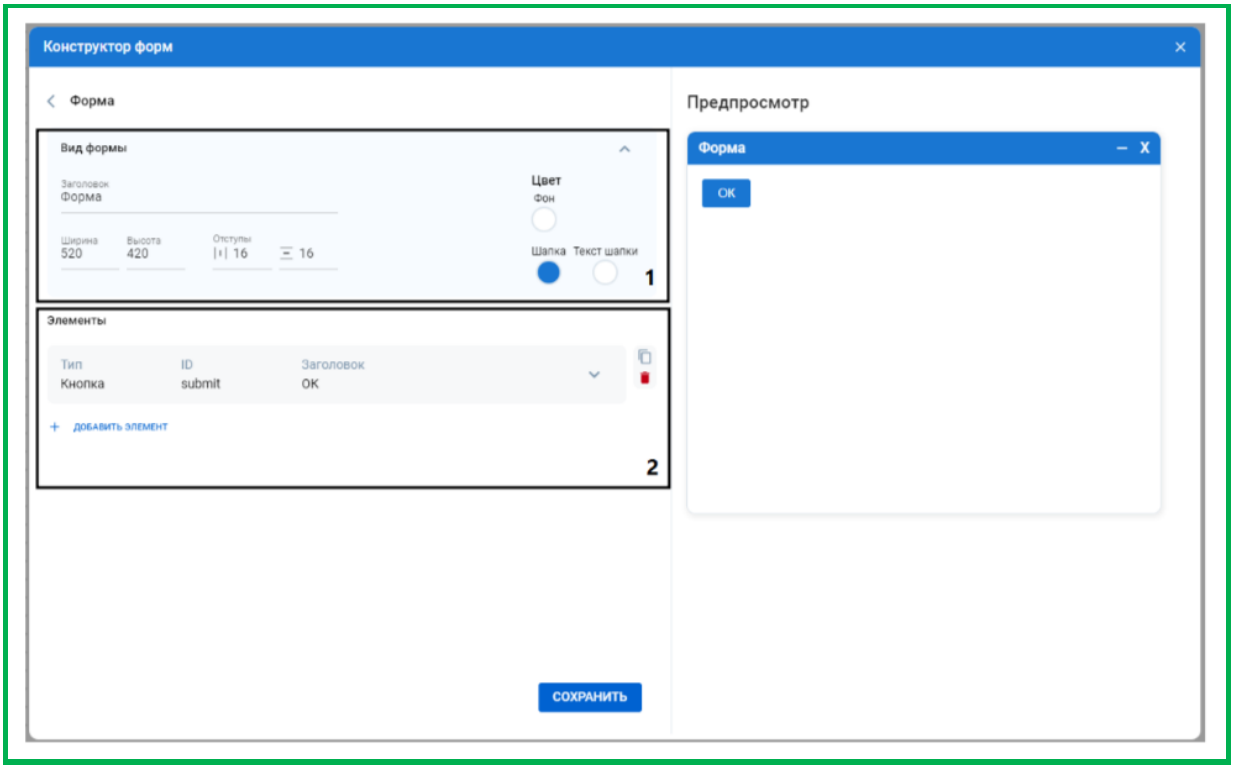
Рассмотрим области работы с пользовательским окном:

1 – Конструктор формы. В левой части конструктора расположены функции для управления формой:

- Создать форму – запуск процесса создания формы.
- Импортировать форму – запускает процесс импорта формы из другого проекта.
- Список форм – список добавленных форм в проекте.

2 – Панель просмотра. В правой части отображается предпросмотр созданной формы.

Для запуска процесса создания формы требуется выполнить клик на кнопку Создать форму. После клика открывается интерфейс изменения формы.



1 – Вид формы

В верхней части располагается раздел Вид формы, позволяющий изменить внешний вид формы:

- Заголовок – заголовок формы; В заголовке окна и элементов допускается использование символов Emojis.
- Ширина и Высота – габариты формы;
- Отступы – отступы элементов и кнопок от края формы;
- Положение кнопок – позиционирование кнопок внутри формы;
- Цвет – цвет фона и шапки формы.

2 – Элементы

В средней части располагается раздел Элементы, который позволяет добавлять на форму интерактивные компоненты ввода и отображения данных. Доступны следующие типы элементов:

- Текст (строка) – однострочное текстовое поле;
- Текст (абзац) – многострочное текстовое поле;
- Пароль – поле для ввода с маскировкой символов;
- Чекбокс – переключатель с двумя состояниями (включено/выключено);
- Селект – выпадающий список с выбором одного значения;
- Файл – элемент для выбора файла;
- Число – поле для ввода числовых значений;
- Дата – элемент выбора даты;
- Радио – группа переключателей для выбора одного варианта из нескольких;
- Кнопка – управляющий элемент формы.

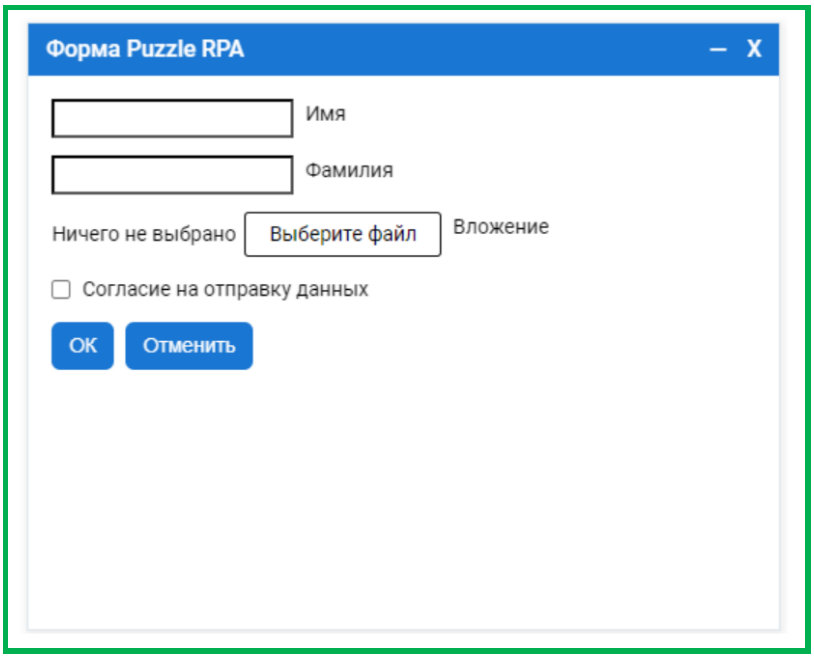
Доступны следующие типы кнопок:

- Подтвердить (ОК) – завершает работу с формой и сохраняет введённые данные; Важно: нажатие кнопки Подтвердить (ОК) приводит к закрытию формы и возврату словаря со значениями полученных от пользователя.
- Отмена – закрывает форму без сохранения изменений; Важно: нажатие кнопки Отмена приводит к закрытию формы и возврату пустого значения от формы.
- Триггер – запускает выполнение заданной функции в проекте;

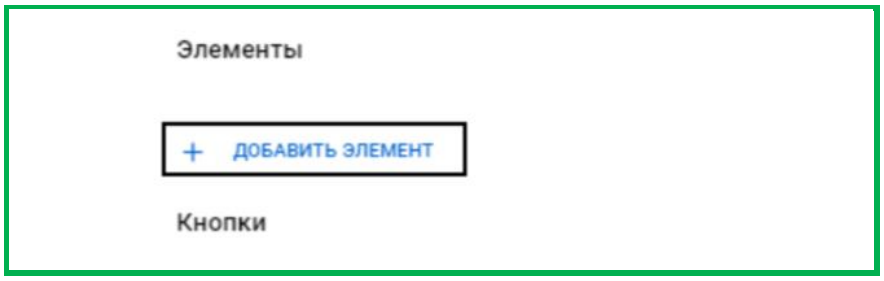
Важно: функцию можно вызвать только из того же процесса, в котором вызывается модальное окно.

- Модальное окно – открывает дополнительное диалоговое окно.

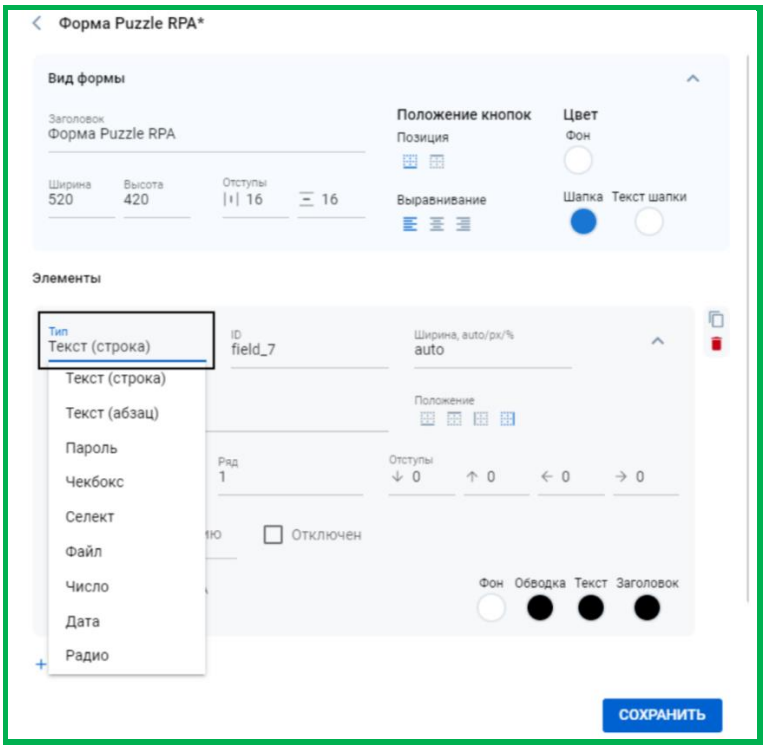
Элементы формы – это отдельные компоненты, из которых состоит пользовательский интерфейс формы. Они позволяют отображать информацию и получать ввод от пользователя.



Для добавления нового элемента формы требуется выполнить клик на кнопку + Добавить элемент.



Конструктор форм позволяет выбрать тип элемента из выпадающего списка. Для этого требуется выполнить клик на поле Тип и выбрать требуемый элемент из выпадающего списка.

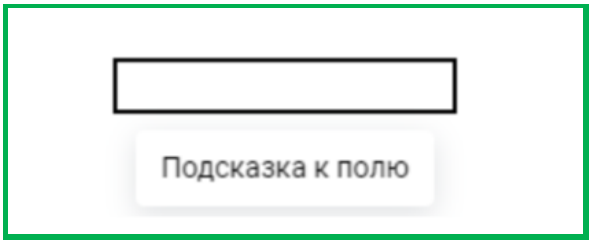


Общие свойства элементов

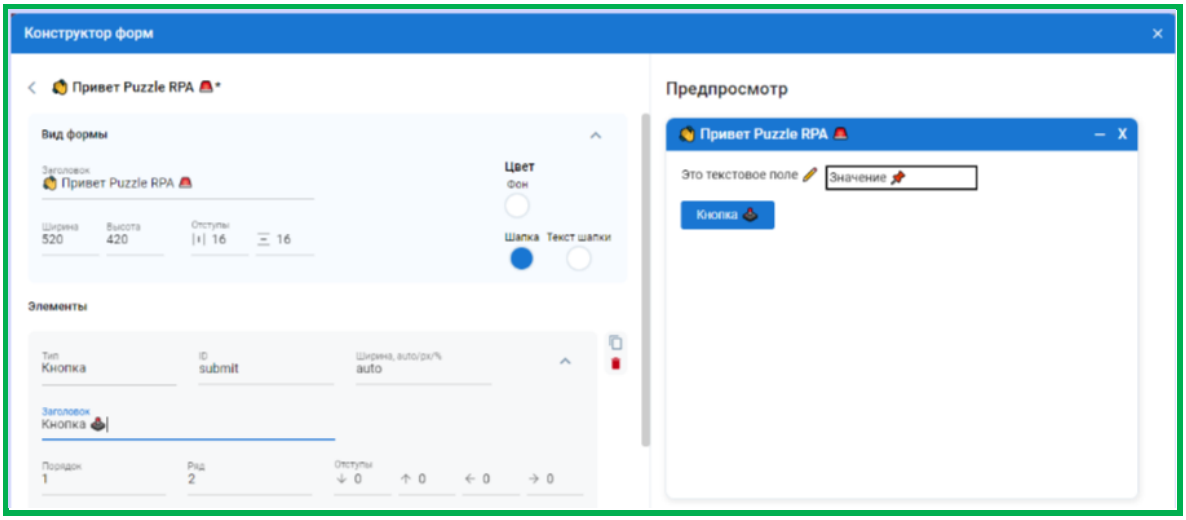
Каждый элемент формы настраивается через набор свойств, которые делятся на общие (применяются ко всем типам элементов) и уникальные (доступны только для отдельных типов).

Общие параметры:

- ID – уникальный идентификатор элемента;
- Ширина – ширина элемента на форме;
- Заголовок – текстовая метка, отображаемая рядом с элементом;
- Положение – положение заголовка относительно элемента;
- Порядок – позиция элемента внутри своего ряда относительно других элементов;
- Ряд – номер строки (горизонтального ряда), в которой размещается элемент (элементы с одинаковым значением ряда располагаются в одной строке);
- Отступ – внешний отступ элемента от соседних компонентов или границ контейнера;
- Значение по умолчанию – начальное значение элемента при открытии формы;
- Отключён – чекбокс, определяющий, недоступен ли элемент для взаимодействия (элемент отображается, но не реагирует на действия пользователя).
- Условия показа – условие для отображения элемента;
- Текст подсказки – текст подсказки, отображаемый рядом с элементом при наведении на него курсора мыши.



Важно: В параметрах: **Заголовок**, **Значение по умолчанию** допускается использование символов Emojis.



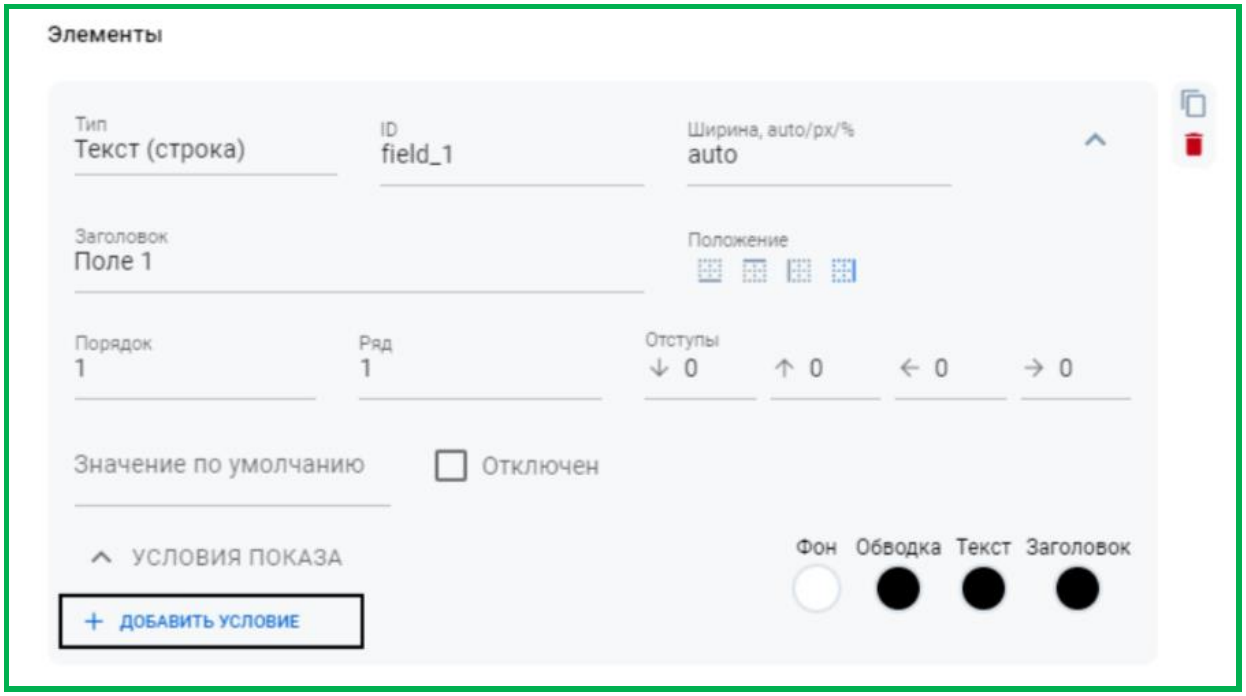
Уникальные параметры:

- Текст (строка): Регулярное выражение – регулярное выражение для валидации данных вводимых в поле (^\d$)

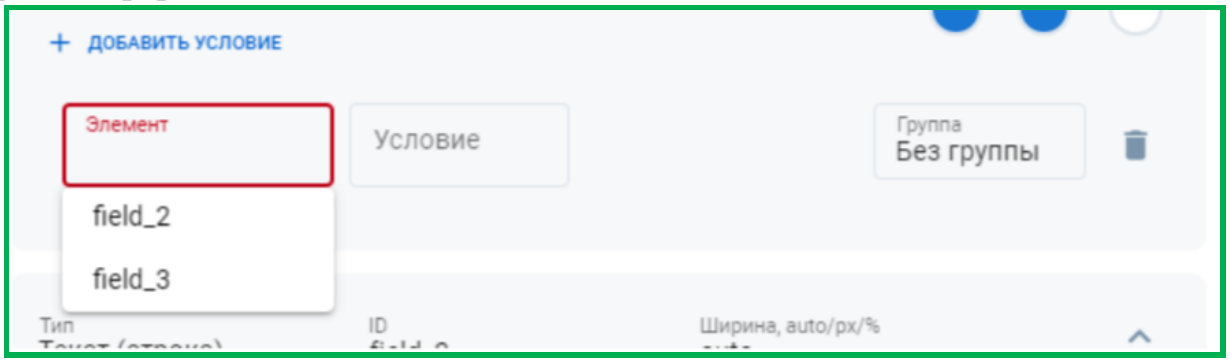
- Текст (абзац): Регулярное выражение – регулярное выражение для валидации данных вводимых в поле (^\d$). Количество строк – количество строк в поле (3).
- Пароль: Регулярное выражение – регулярное выражение для валидации данных вводимых в поле (^\d$)
- Селект: Варианты – варианты значений через запятую (Вариант 1, Вариант 2)
- Файл: Множественный выбор – чексбокс, позволяющий пользователям выбрать несколько файлов. Выбор папки – чексбокс, позволяющий пользователям выбрать папку. Типы файлов – расширения доступных для выбора файлов через запятую (pdf,docx,txt)
- Число: Мин – минимальное значение. Макс – максимальное значение. Шаг – шаг переключения с помощью кнопок увеличения/уменьшения значения
- Радио: Варианты – варианты значений через запятую (Вариант 1, Вариант 2)

Условия показа. Для каждого элемента доступен выбор условия показа в зависимости от значения в другом элементе. Для этого требуется:

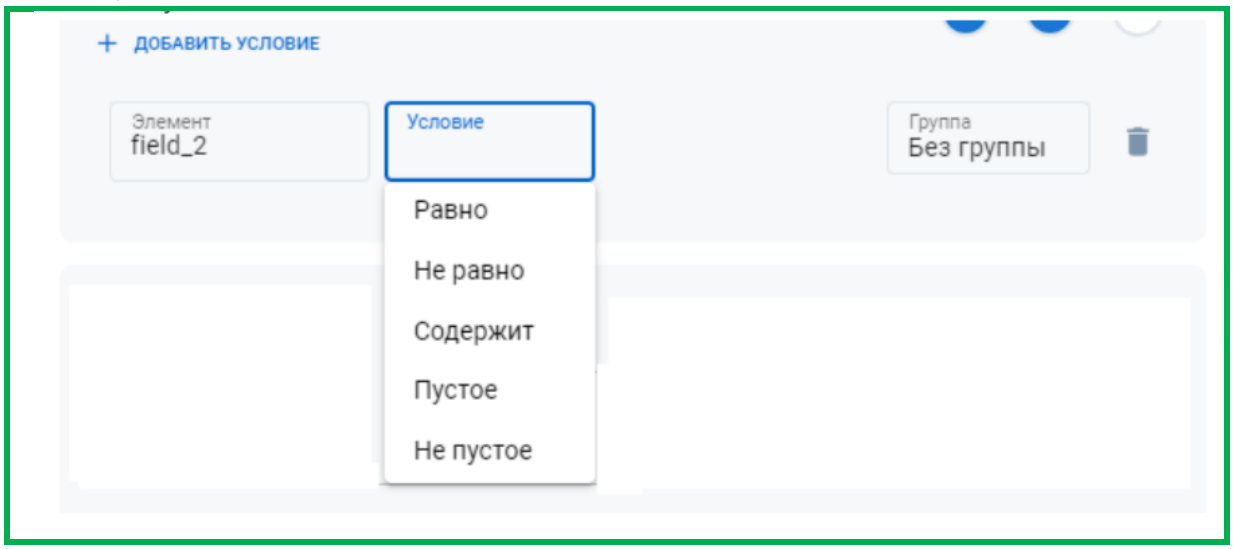
- Выполнить клик на кнопку + Добавить условие;



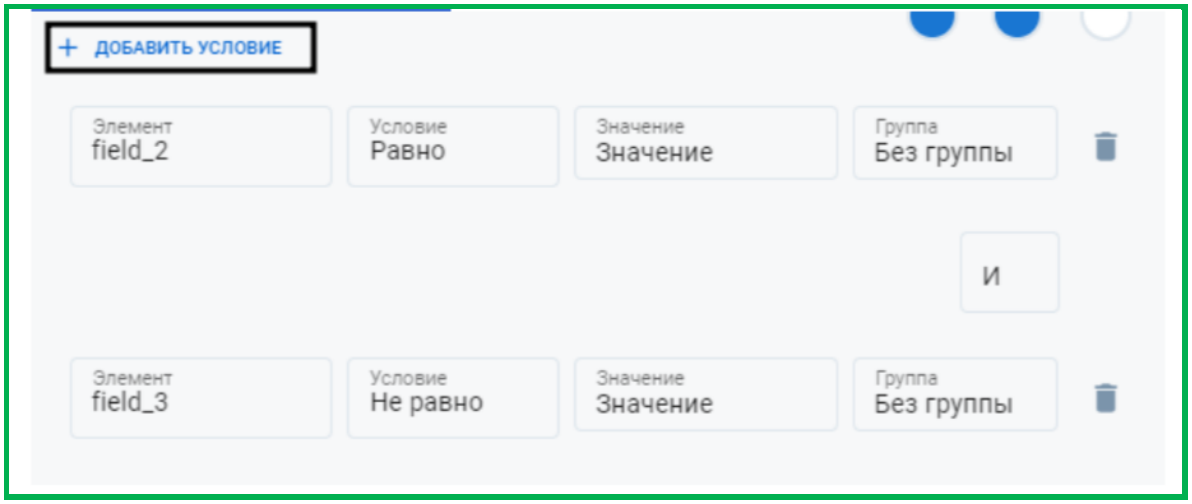
- Выбрать элемент, в зависимости от значения которого будет отображаться выбранная форма;



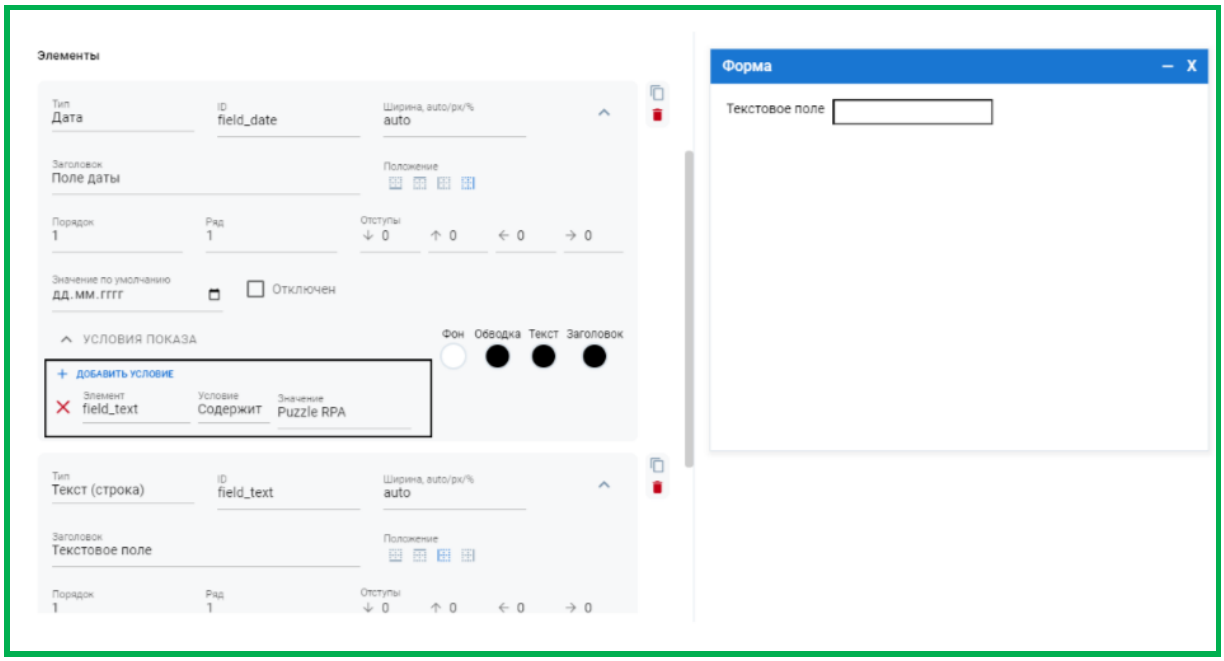
• Указать условие;



• При необходимости добавить дополнительные условия, выполнив клик на кнопку + **Добавить условие**.

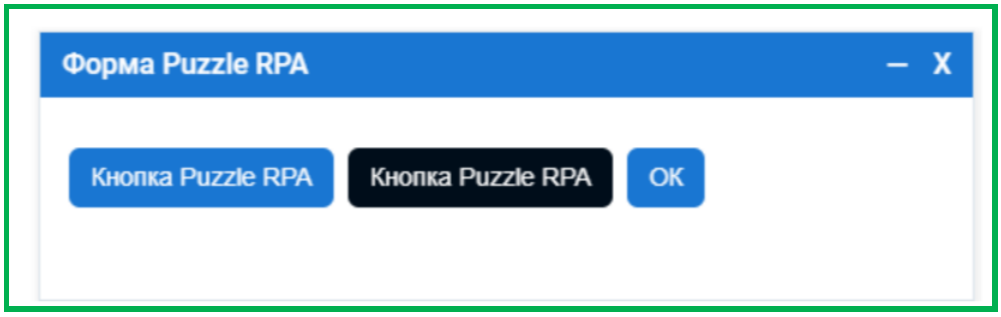


Пример условия показа: элемент с ID `field_date` (Поле даты) будет отображен только, если в текстовом поле с ID `field_text` (Текстовое поле) будет значение *Puzzle RPA*.



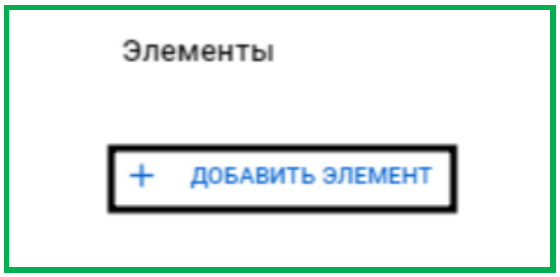
Кнопки формы

Кнопки формы – это управляющие элементы, предназначенные для инициации действий: подтверждения ввода, отмены операции, запуска процессов или открытия дополнительных интерфейсов.



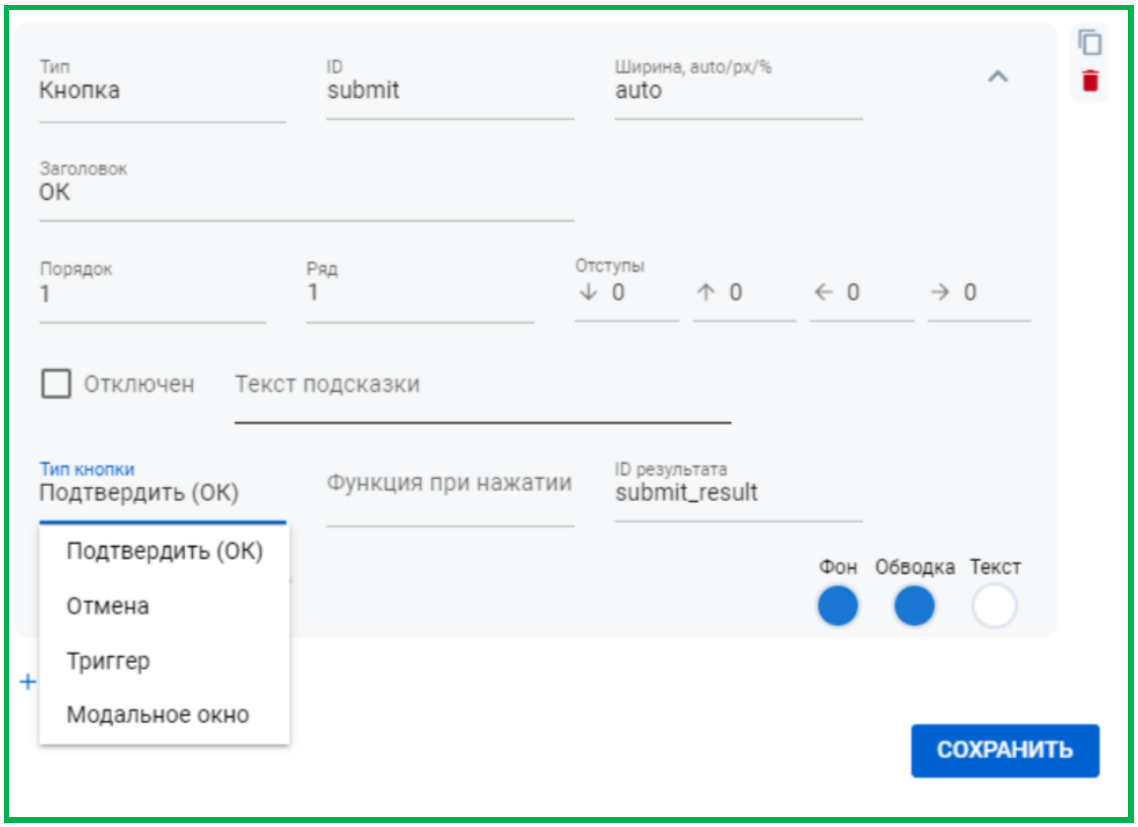
Добавление кнопки

Для добавления новой кнопки на форму требуется выполнить клик на кнопку + Добавить элемент.



Тип кнопки

Конструктор форм позволяет выбрать тип кнопки из выпадающего списка. Для этого требуется выполнить клик на поле Тип и выбрать требуемый тип из списка.



Общие свойства кнопок

Каждая кнопка формы настраивается через набор свойств, которые делятся на общие (применяются ко всем типам кнопок) и уникальные (доступны только для отдельных типов).

Общие параметры:

- Ширина – ширина кнопки на форме; Поддерживает следующие форматы значения: auto – ширина определяется автоматически на основе содержимого и контекста; числовое значение с суффиксом px (например, 120px) – фиксированная ширина в пикселях; числовое значение с суффиксом % (например, 50%) – ширина задаётся как процент от ширины контейнера формы.
- Заголовок – текст, отображаемый на кнопке;
- Порядок – позиция кнопки внутри своего ряда относительно других элементов;
- Функция при нажатии – выпадающий список, позволяющий выбрать функцию из доступных в проекте, которая будет автоматически вызвана при нажатии на кнопку.

Уникальные свойства кнопок. При выборе типа кнопки **Модальное окно** необходимо указать:

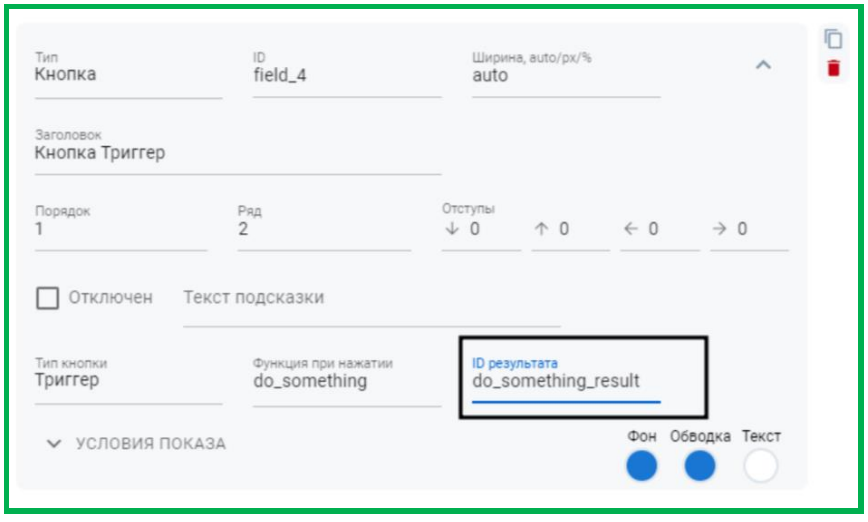
- **Файл** – выпадающий список, позволяющий выбрать другую форму из списка созданных форм в данном проекте.
- **ID поля** – идентификатор, используемый в качестве ключа в словаре результата при возврате данных из дочернего модального окна (когда текущая форма вызывает другое модальное окно, значение, возвращённое этим окном, записывается в результат по указанному ID поля).

Кнопка Триггер

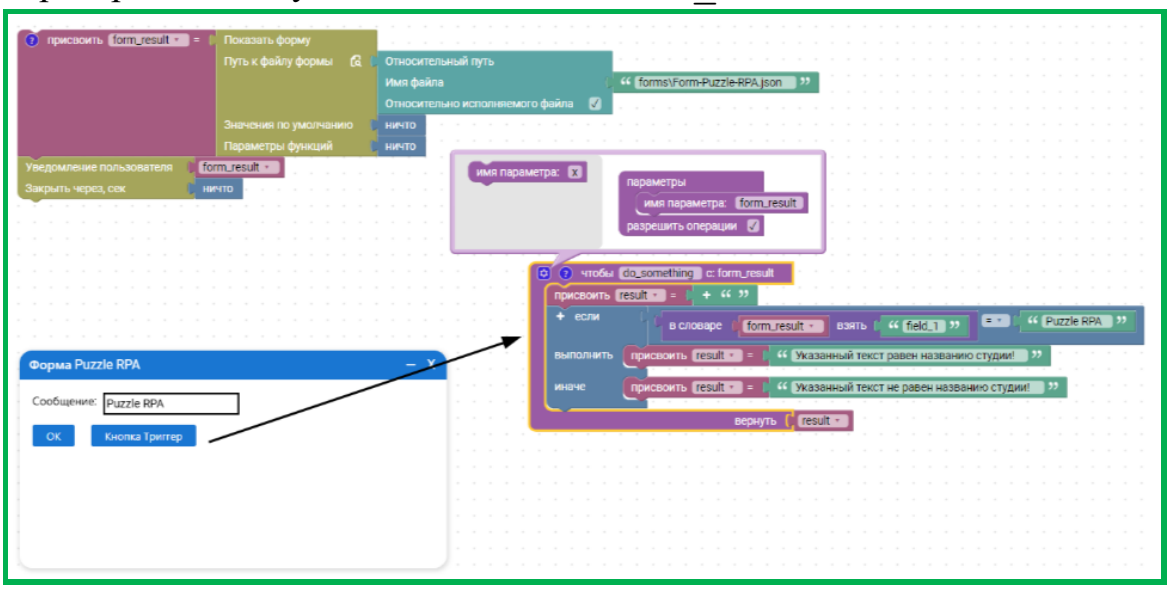
Кнопка **Триггер** – позволяет запустить функцию. Вызов функции не вызывает закрытие или завершение работы формы.

Необходимо указать:

- Тип кнопки – Триггер;
- **Функция при нажатии** – созданная функция в проекте, которая будет вызвана;
- **ID результата** – уникальный идентификатор, значению которого будет присвоен результат работы функции.



В данном примере кнопка Триггер вызывает функцию `do_something`, которая проверяет текст, указанный в поле `field_1`.



Важно: При использовании параметров в функции важно всегда указывать первый параметр (например, с именем `form_result`), через который будет передаваться словарь с данными, полученными из формы.

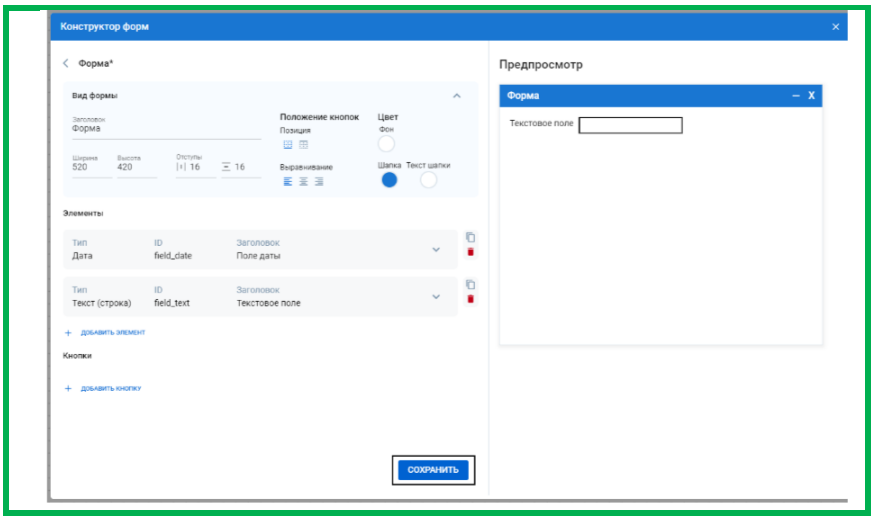
Результат возвращается при завершении работы формы (нажатии кнопки **ОК**) с ключом **do_something_result**, указанным в настройках кнопки.

```
{'field_1': 'Puzzle RPA',
'do_something_result': 'Указанный текст равен
названию студии!'}
```

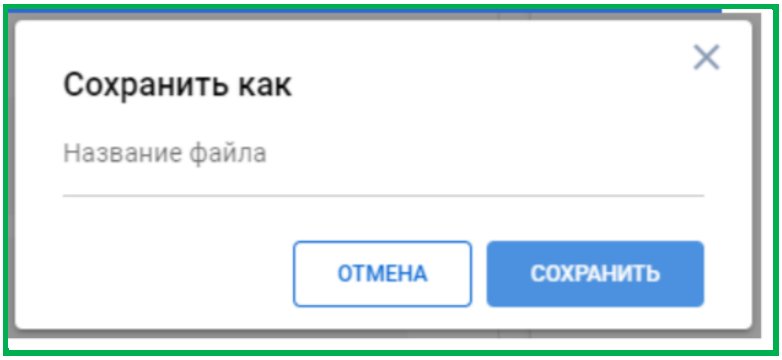
Сохранение формы

Конструктор форм позволяет сохранить созданную форму. Для этого требуется:

- Выполнить клик на кнопку Сохранить;



• Указать название файла и выполнить клик на кнопку Сохранить.

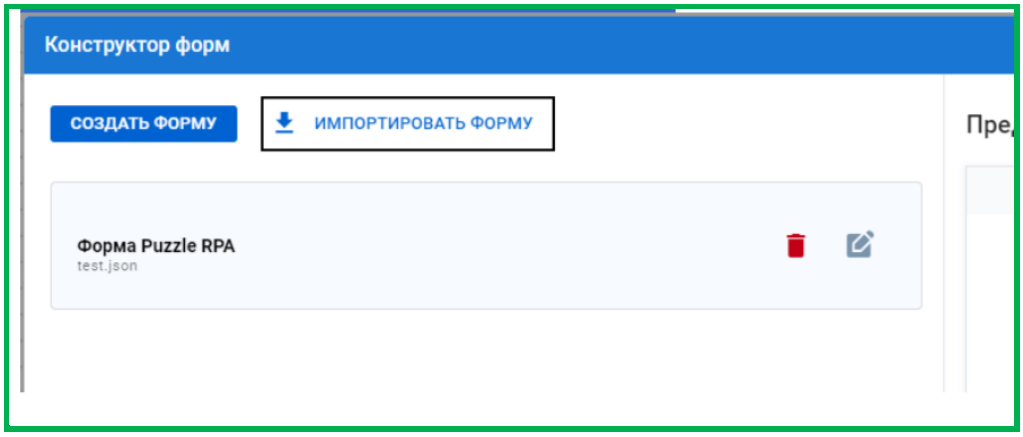


Важно: Формы сохраняются в папку проекта в директорию: *директория_проекта\resource\forms*

Импорт формы

Конструктор форм позволяет импортировать форму, созданную в другом проекте. Для этого требуется:

• Выполнить клик на кнопку Импортировать форму;

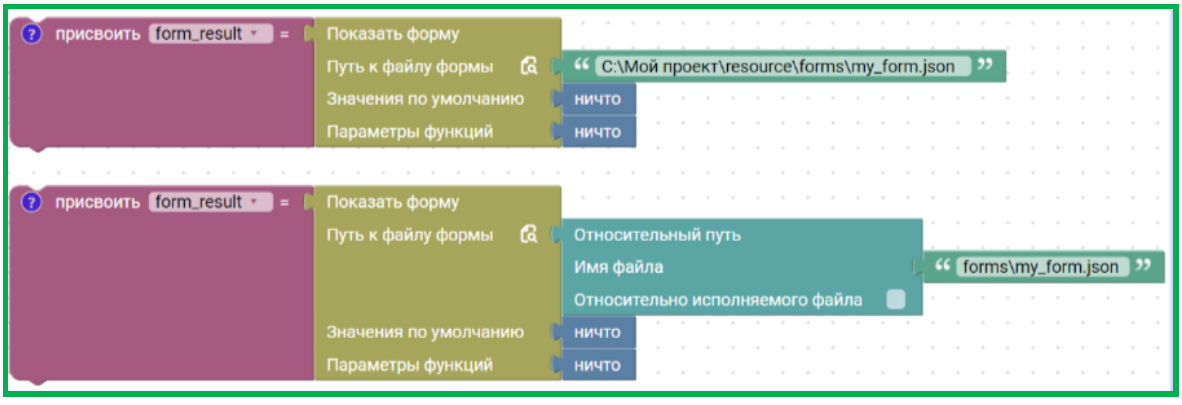


• Открыть форму из другого созданного проекта в студии Puzzle RPA.

Вызов формы

Вызов созданной формы в алгоритме программного робота осуществляется с помощью блока Показать форму. Для этого необходимо указать:

• Путь к файлу формы – путь до файла сохраненной формы. В поле путь до файла требуется указать путь до сохраненного файла формы. Можно указать как прямой путь, так и с использованием блока Относительный путь. В данном примере вызывается форма из файла my_form.json двумя способами:



• Значения по умолчанию – словарь для изменения параметров элементов формы. Блок позволяет динамически изменять свойства созданных элементов в форме. Для этого необходимо указать в блоке словарь:

```
{
    'id_element': {
                    'property': 'new_value'
                  }
}
```

• Параметры функции–значения параметров вызываемой функции.

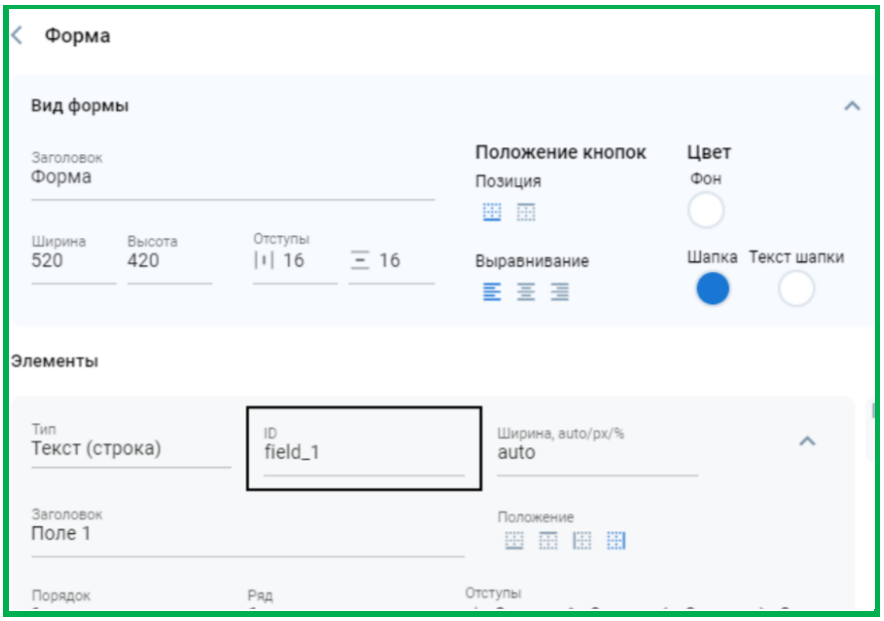
Пример динамического изменения параметров. В данном примере выполняется запуск формы и изменение свойств текстового поля с ID field_1:

• label – изменение заголовка поля на “Поле отключено”;

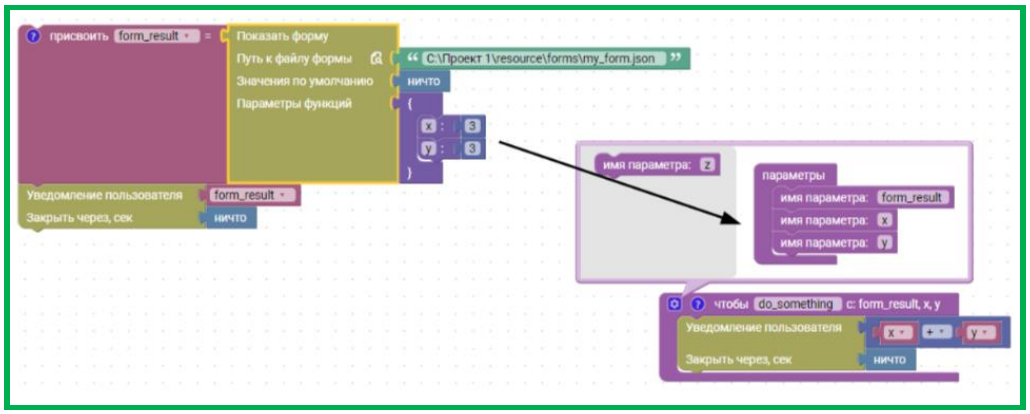
• disabled – изменение признака отключения поля на “Истина”.



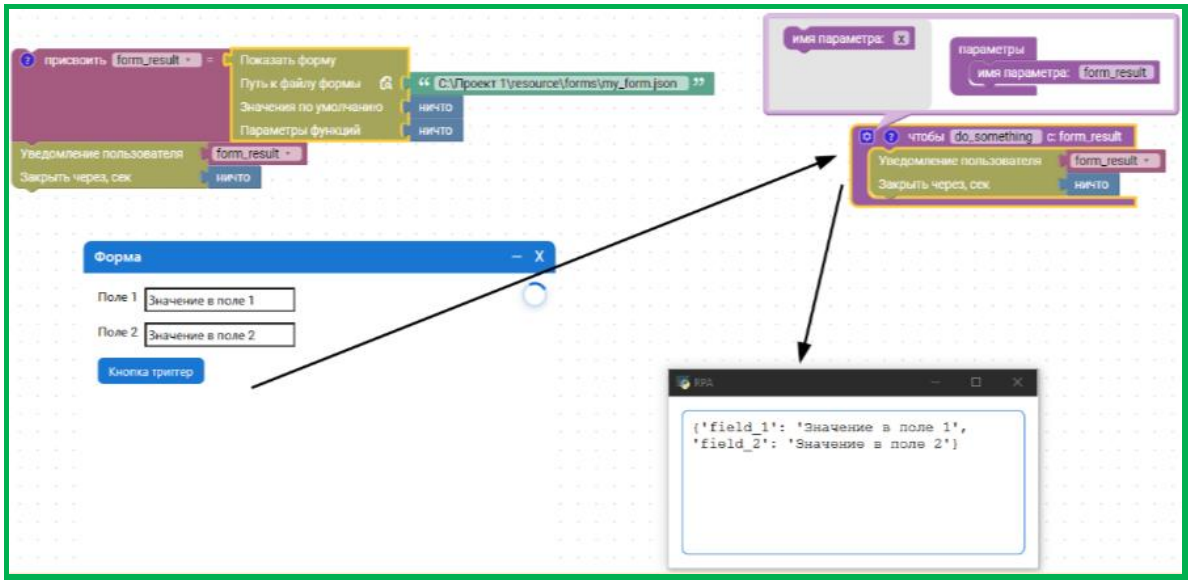
Важно: Ключ в словаре field_1 – это ID элемента на форме. Значение должно соответствовать значению в поле ID изменяемого элемента:



Если в форме используется кнопка Триггер для вызова функции, то при этом можно передавать параметры в вызываемую функцию.



Важно: При использовании параметров в функции важно всегда указывать первый параметр (например, с именем form_result), через который будет передаваться словарь с данными, полученными из формы.



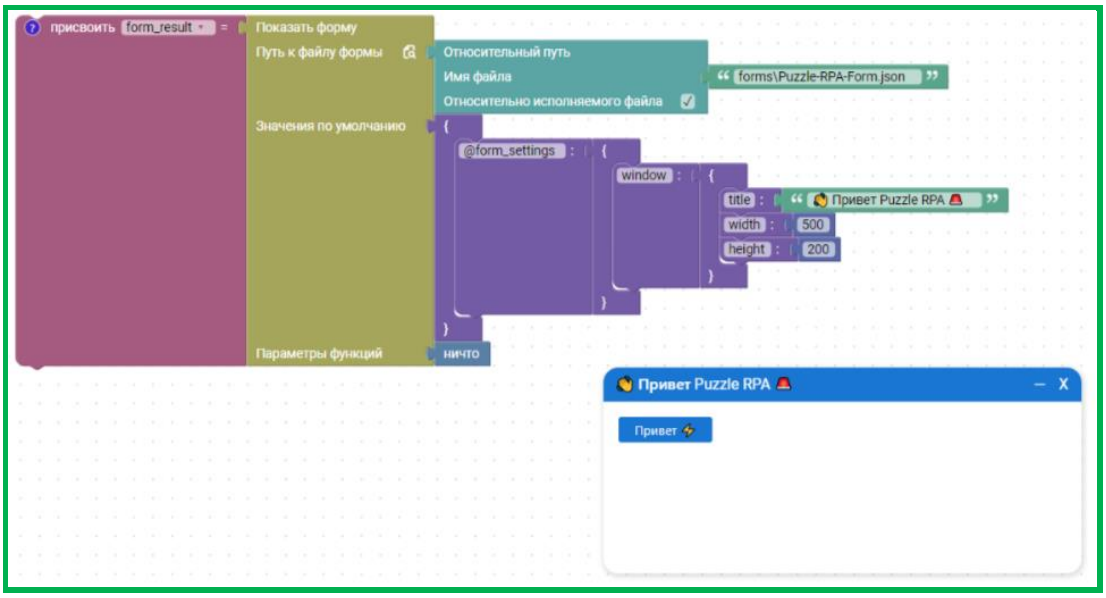
Результат выполнения формы

Форма возвращает результат только при нажатии кнопки типа Подтвердить (OK). В этом случае результатом является словарь, где ключи соответствуют ID элементов формы, а значения – данным, введённым или выбранным пользователем. Если пользователь завершает работу с формой с помощью кнопки типа Отмена, форма возвращает значение “Ничто”.

Доступные параметры для динамического изменения

Элементы имеют ряд параметров, которые можно изменять динамически во время вызова формы из алгоритма:

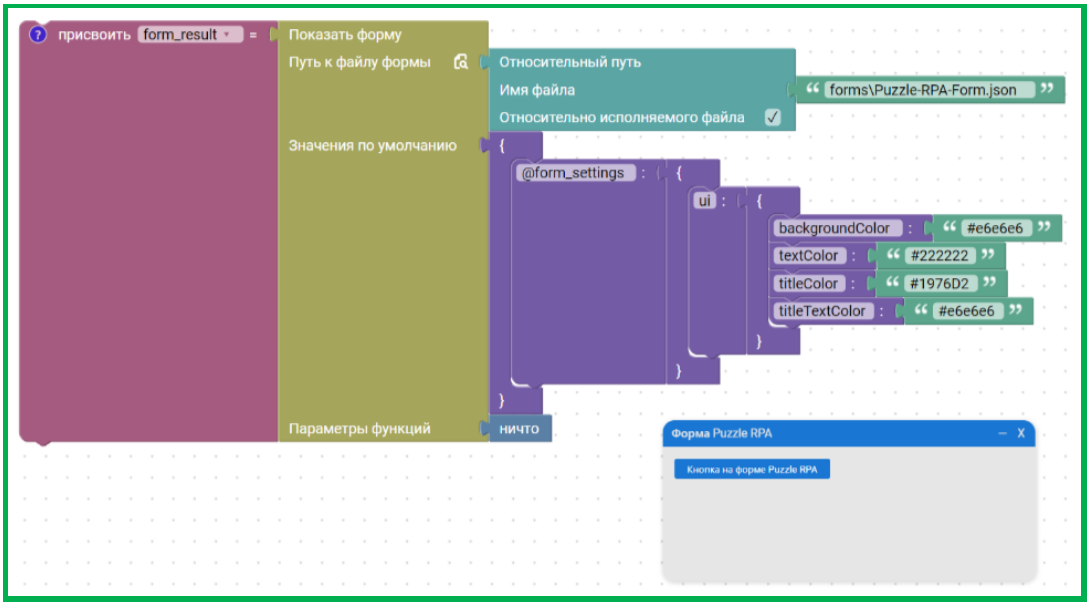
- Заголовок окна (window)
  - ✓ `title` – заголовок окна (текстовое значение);
  - ✓ `width` – ширина окна (числовое значение);
  - ✓ `height` – высота окна (числовое значение).



Параметры формы передаются в виде значения ключа словаря `@form_settings`.

```
{
    "@form_settings": {
        "window":{
            "title": "👋 Привет Puzzle RPA 🚨",
            "width": 500,
            "height": 200,
        }
    }
}
```

- Внешний вид (ui)
  - ✓ `backgroundColor` – цвет фона формы (текстовое значение);
  - ✓ `textColor` – цвет текста формы (текстовое значение);
  - ✓ `titleColor` – цвет заголовка формы (текстовое значение);
  - ✓ `titleTextColor` – цвет текста заголовка формы (текстовое значение).



Параметры формы передаются в виде значения ключа словаря `@form_settings`.

```
{
    "@form_settings": {
        "ui":{
            "backgroundColor": "#e6e6e6",
            "textColor": "#222222",
            "titleColor": "#1976D2",
            "titleTextColor": "#e6e6e6",
        }
    }
}
```

- Текст (строка)
  - ✓ `label` – заголовок поля (текстовое значение);
  - ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
  - ✓ `default` – значение по умолчанию (текстовое значение);
  - ✓ `tooltipText` – текст подсказки элемента (текстовое значение).
- Текст (абзац)
  - ✓ `label` – заголовок поля (текстовое значение);
  - ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
  - ✓ `default` – значение по умолчанию (текстовое значение);
  - ✓ `rows` – количество строк (числовое значение);
  - ✓ `tooltipText` – текст подсказки элемента (текстовое значение).
- Пароль
  - ✓ `label` – заголовок поля (текстовое значение);
  - ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
  - ✓ `tooltipText` – текст подсказки элемента (текстовое значение).
- Чекбокс
  - ✓ `label` – заголовок поля (текстовое значение);
  - ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
  - ✓ `default` – признак, определяющий, выбран ли чекбокс при открытии формы (допустимое значение: Истина/Ложь);
  - ✓ `tooltipText` – текст подсказки элемента (текстовое значение).
- Селект
  - ✓ `label` – заголовок поля (текстовое значение);

- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `default` – значение по умолчанию (текстовое значение);
- ✓ `options` – список значений в выпадающем списке (массив значений);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

• Файл

- ✓ `label` – заголовок поля (текстовое значение);
- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `allowMultiple` – признак, определяющий, доступен ли множественный выбор файлов (допустимое значение: Истина/Ложь);
- ✓ `directory` – признак, определяющий, требуется выбрать папку (допустимое значение: Истина/Ложь);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

• Число

- ✓ `label` – заголовок поля (текстовое значение);
- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `default` – значение по умолчанию (числовое значение);
- ✓ `min` – минимальное значение (числовое значение);
- ✓ `max` – максимальное значение (числовое значение);
- ✓ `step` – шаг переключения между числами (числовое значение);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

• Дата

- ✓ `label` – заголовок поля (текстовое значение);
- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `default` – значение по умолчанию (текстовое значение в формате YYYY –MM –DD. Пример: 2025 –10 –08);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

• Радио

- ✓ `label` – заголовок поля (текстовое значение);
- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `default` – значение по умолчанию (текстовое значение);
- ✓ `options` – список значений в выпадающем списке (массив значений);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

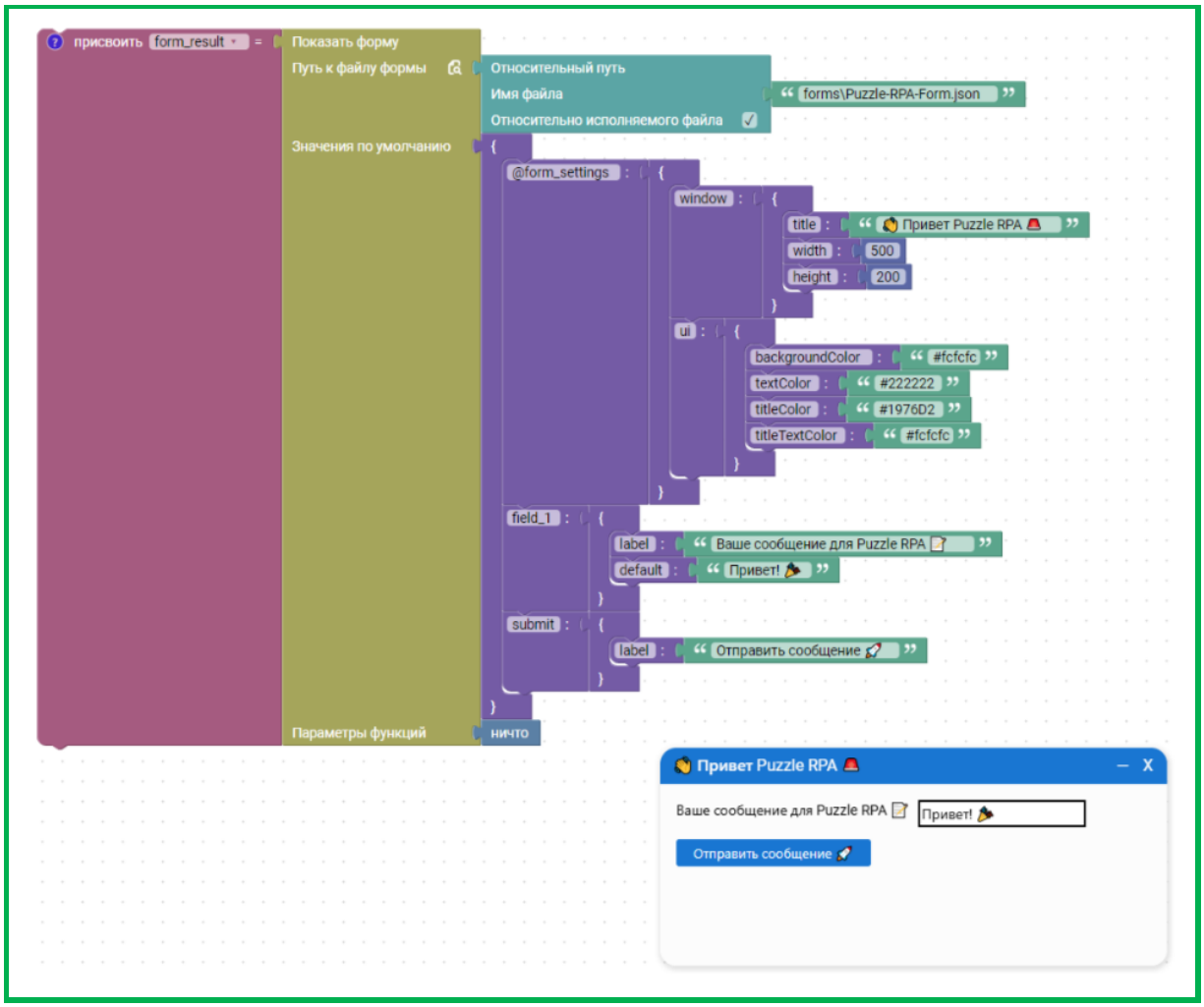
• Кнопка

- ✓ `label` – заголовок поля (текстовое значение);
- ✓ `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь);
- ✓ `tooltipText` – текст подсказки элемента (текстовое значение).

Пример динамического изменения

В данном примере:

- Устанавливается заголовок окна *Привет Puzzle RPA* и изменяется размер окна;
- Изменяются цвета формы (`ui`);
- Для текстового поля `field_1` устанавливается заголовок *Ваше сообщение для Puzzle RPA* и значение по умолчанию *Привет!*;
- Для кнопки `submit` изменяется заголовок *Отправить сообщение*.

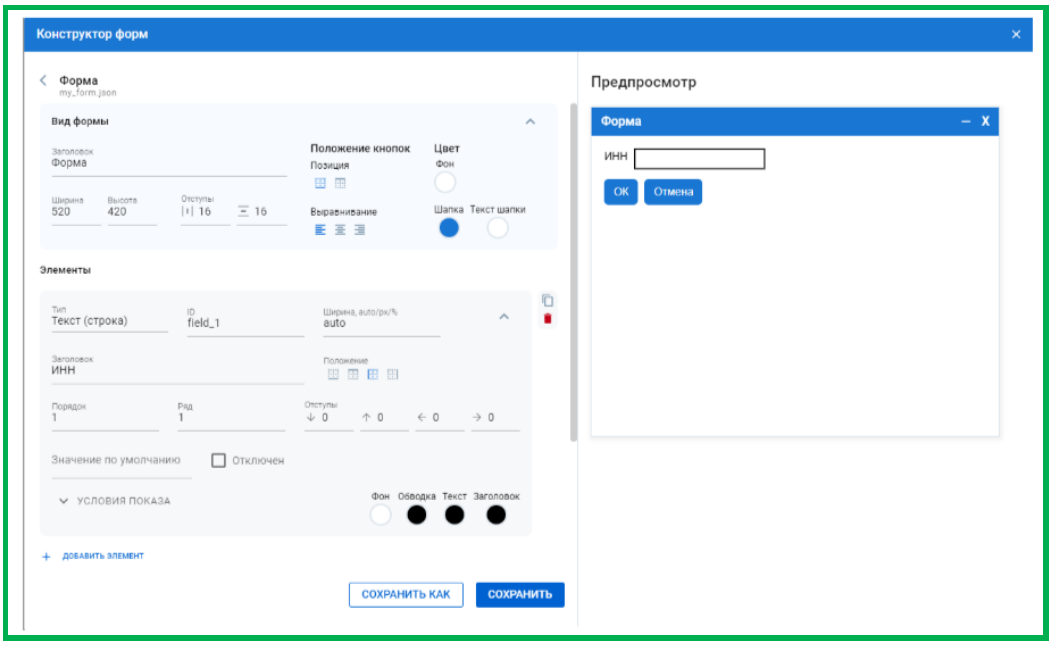


ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

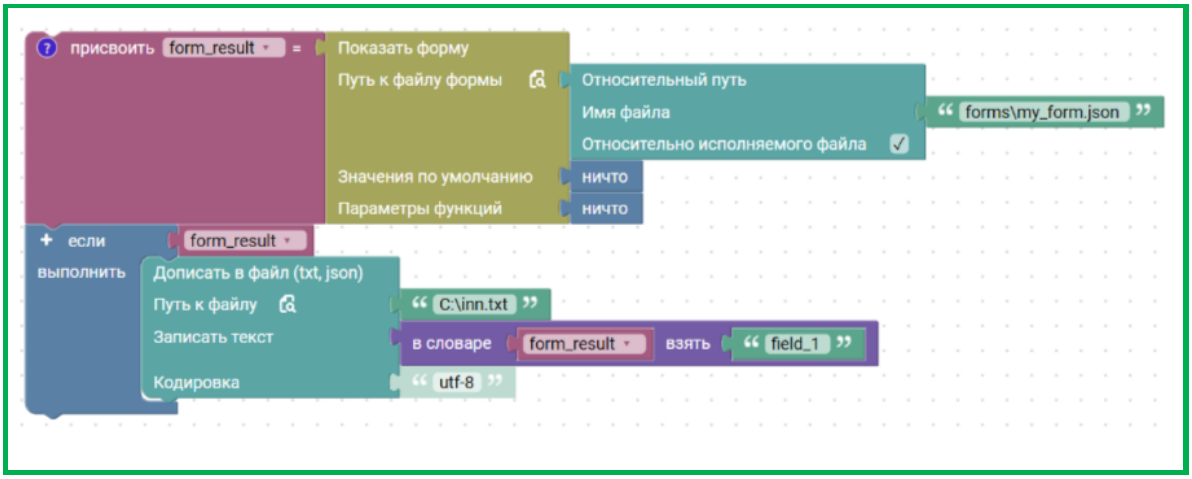
Пример 1 – Получение данных от пользователя

В данном примере выполняется получение идентификационного номера налогоплательщика (ИНН) от пользователя для использования в алгоритме программного робота. Для этого:

- Создается форма с текстовым полем для ввода ИНН кнопкой подтверждения или отмены ввода. Элемент в форме для ввода – это текстовое поле. Данный элемент имеет ID `field_1`. Данный ID требуется для получения данных из формы.



- Создается алгоритм программного робота состоящий из:
  - ✓ Вызов созданной формы;
  - ✓ Проверка на пустое значение, полученное от формы (если пользователь нажал на кнопку Отмена, форма возвращает “Ничто”);
  - ✓ Если пользователь выполнил нажатие на кнопку ОК и отправил данные, выполняется запись полученного ИНН в файл. Значение из поля ИНН извлекается по ID `field_1` элемента на форме.



Пример 2 – Кнопка Триггер

Тип кнопки Триггер позволяет запустить функцию

В данном примере выполняется запуск двух функций с помощью кнопок на форме. Для этого:

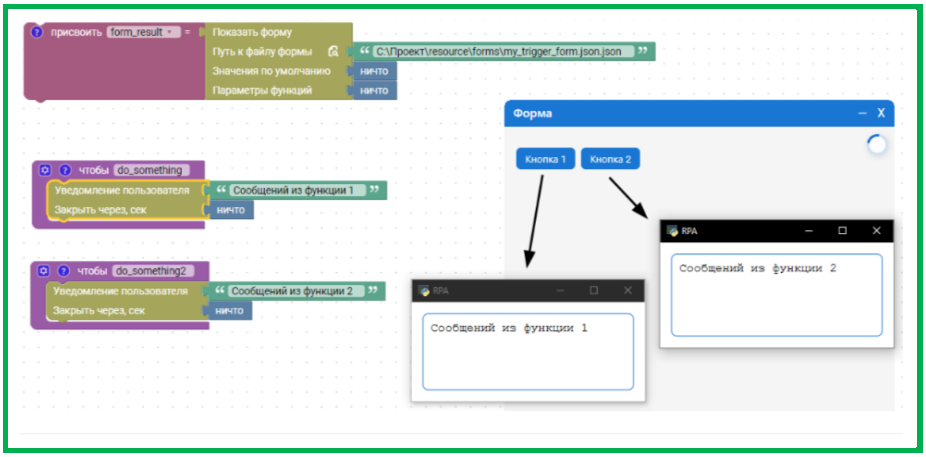
- Создается кнопка с типом Триггер;
- Задается Действие при нажатии для созданной кнопки и выбирается функция из проекта `do something`;



- Создается кнопка номер 2 с типом Триггер;
- Задается Действие при нажатии для созданной кнопки и выбирается функция из проекта `do something2`.



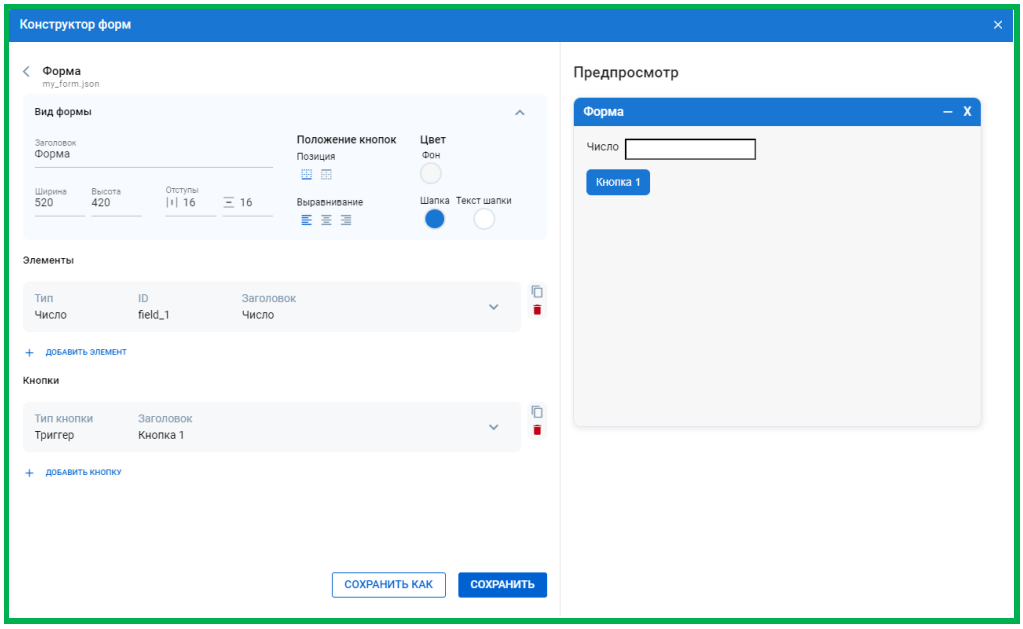
При вызове формы, кнопки на форме с типом Триггер будут запускать выполнение функции `do something` или `do something2`. Ограничение: функцию можно вызвать только из того же процесса, в котором вызывается модальное окно.



Пример 3 – Передача параметров функции

В этом примере демонстрируется вызов функции с передачей параметров. Для этого:

1. Создаётся форма с числовым полем с ID `field_1` и кнопкой **Триггер**, привязанной к функции `do_something`:

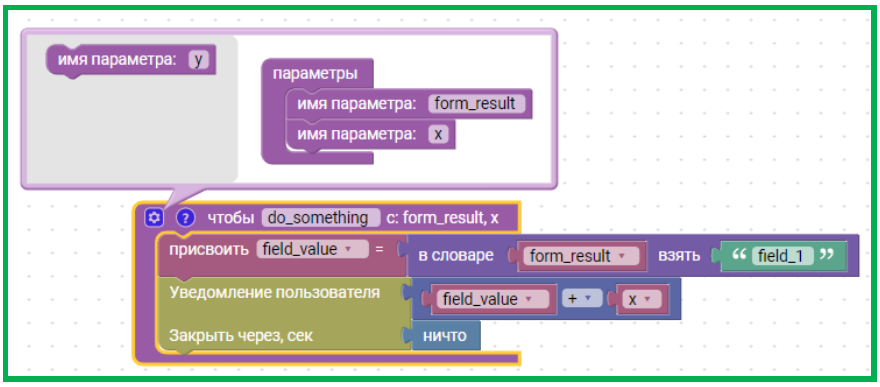


2. Создаётся функция, принимающая два параметра:

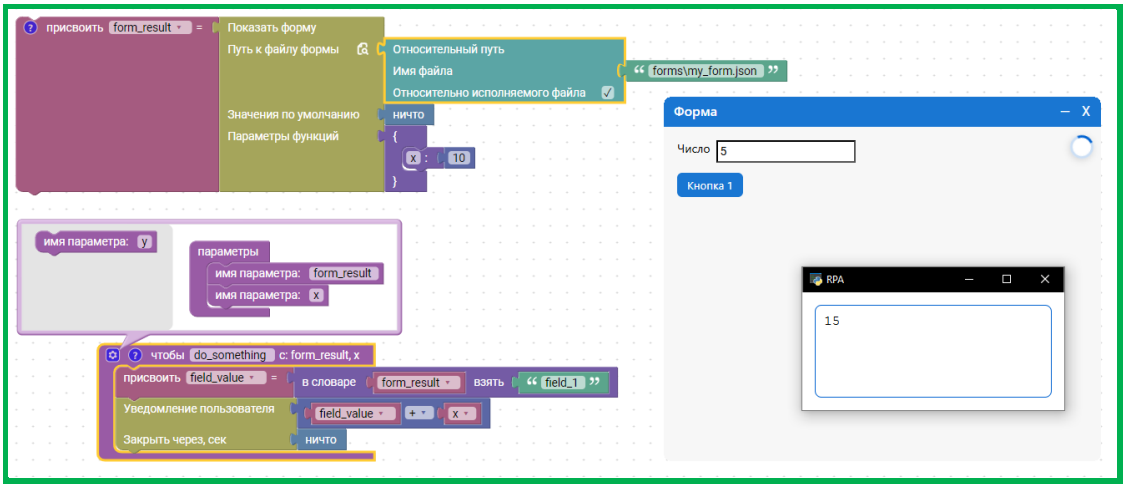
- `form_result` – словарь с данными, полученными из формы;
- `x` – дополнительный параметр, значение которого задаётся при вызове формы.

Из `form_result` извлекается число из поля `field_1`, после чего оно складывается со значением параметра `x`. Результат операции выводится на экран.

Важно: При использовании параметров функции первым аргументом всегда должен быть параметр, принимающий результаты заполнения элементов формы (словарь с данными). Дополнительные параметры указываются опционально, и их значения необходимо задавать в поле **Параметры функции** при вызове формы.



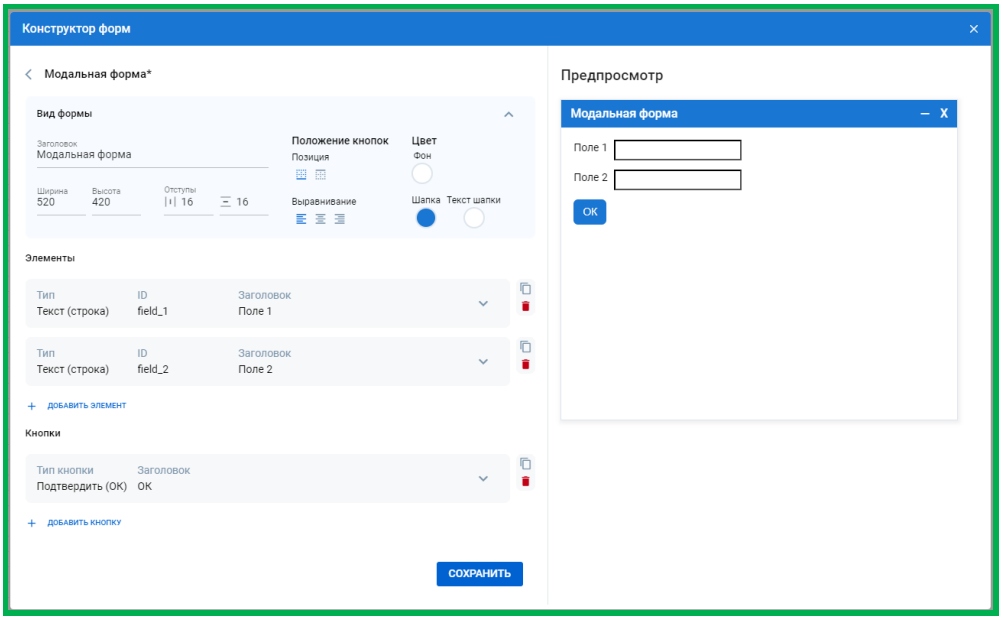
После запуска формы выполняется сложение значения 5 из числового поля `field_1` и числа 10, переданного в параметре `x` при вызове формы.



Пример 4 – Вызов дочернего модального окна

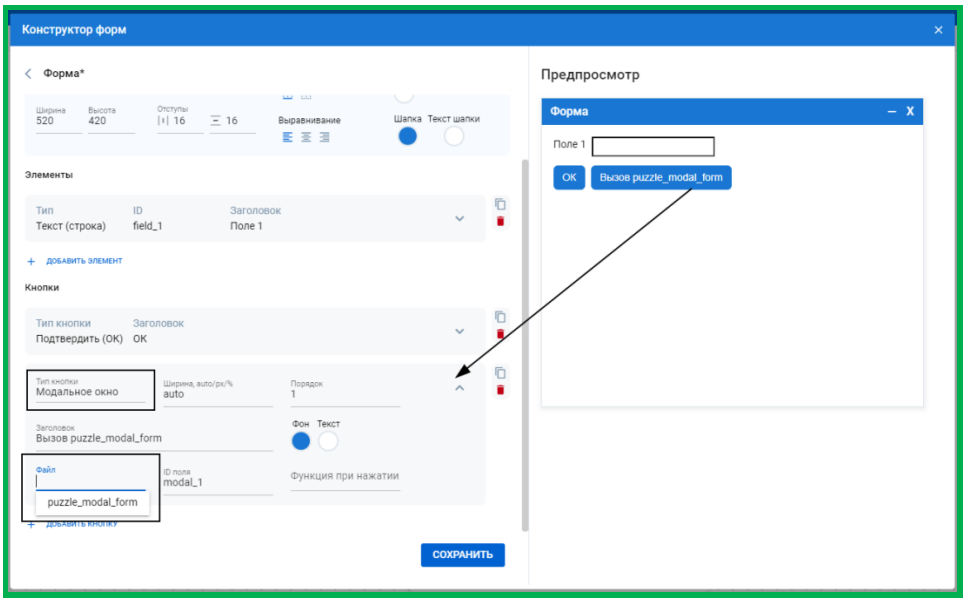
В данном примере выполняется запуск дочернего модально окна. Для этого:

1. Создается модальное окно с названием `puzzle_modal_form`:

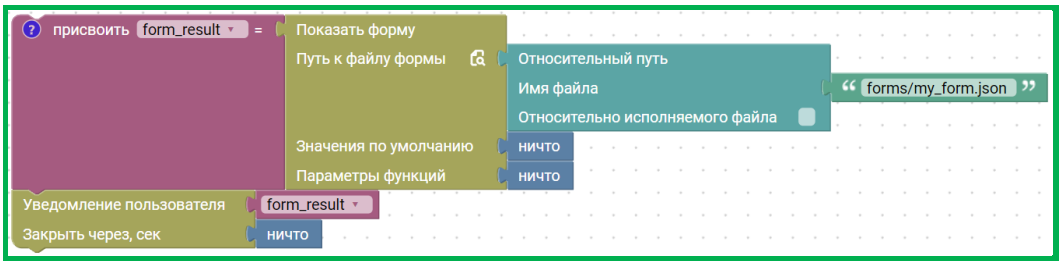


2. Создается вторая форма:

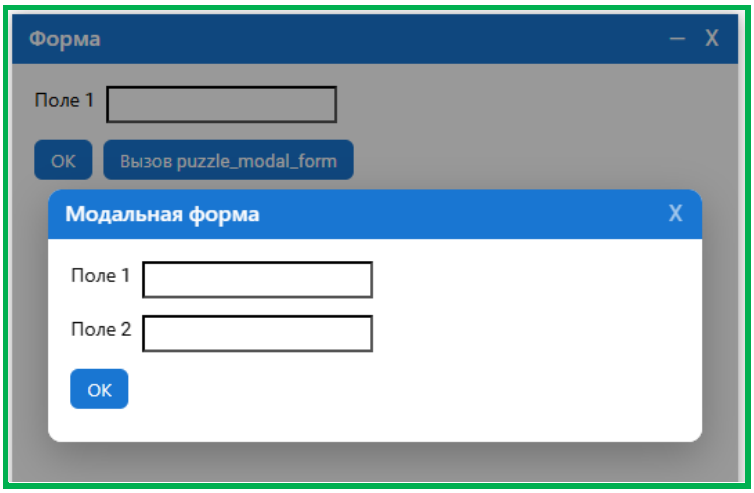
В данной форме создаётся кнопка типа **Модальное окно**, при нажатии на которую открывается уже созданная форма `puzzle_modal_form`.



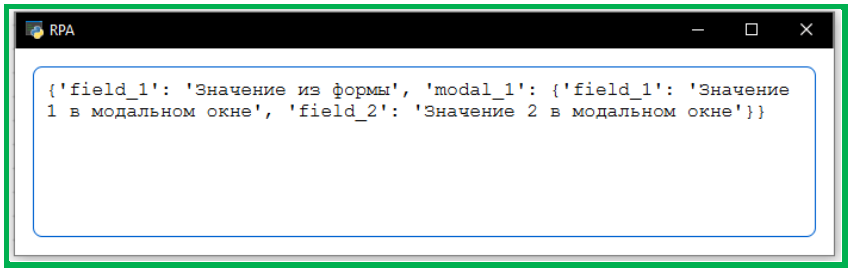
3. Создается вызов формы:



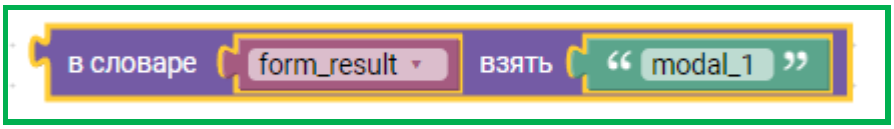
При клике на кнопку **Вызов** `puzzle_modal_form` открывается дополнительная форма, созданная на шаге 1.



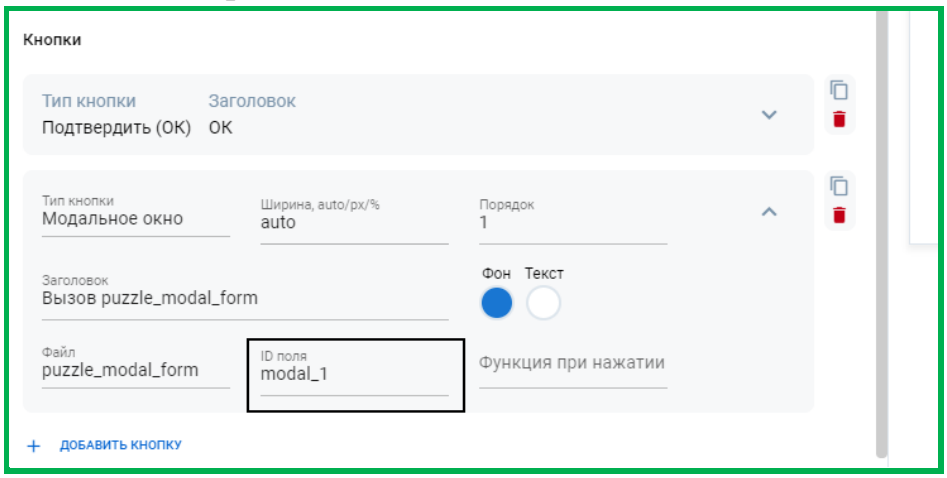
Результат будет получен в виде словаря:



Значение из модального окна `puzzle_modal_form` можно получить по ключу словаря `modal_1`:



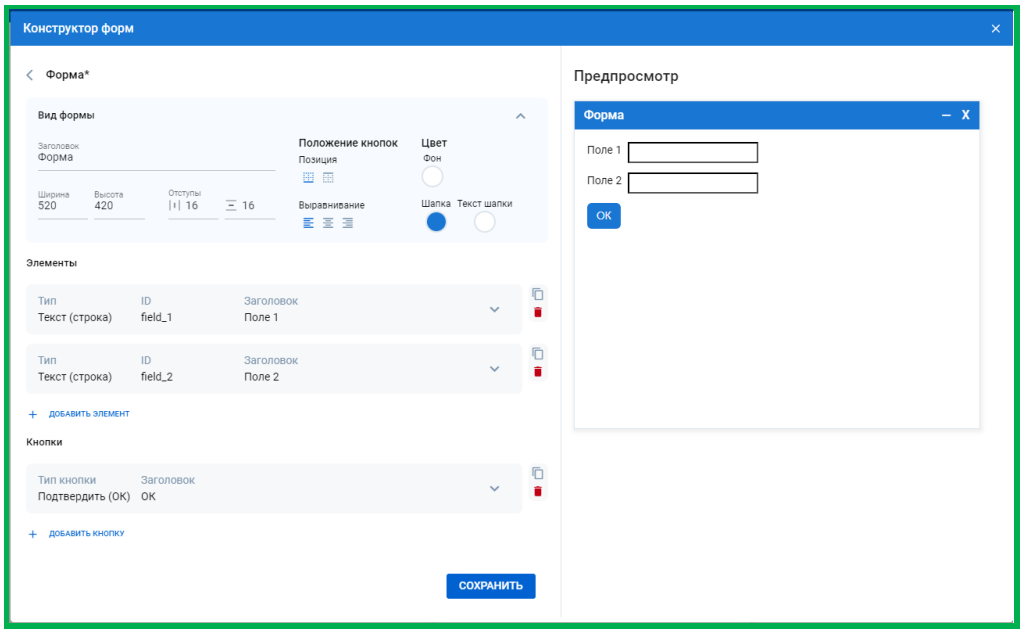
Ключ задается в настройках кнопки вызова модального окна:



Пример 5 – Изменение свойств элементов при вызове формы

В данном примере выполняется отключение текстового поля при вызове формы. Для этого:

1. Создается форма с двумя текстовыми полями с ID `field_1` и `field_2`:

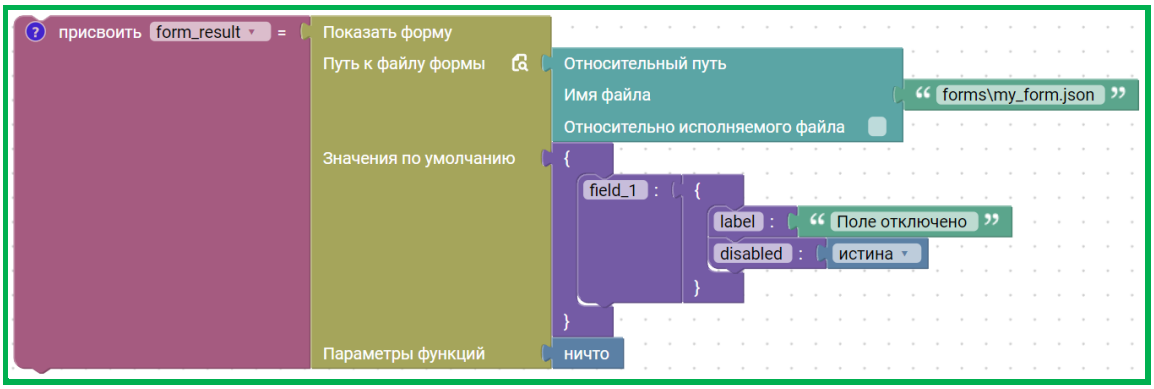


Создается вызов созданной формы:

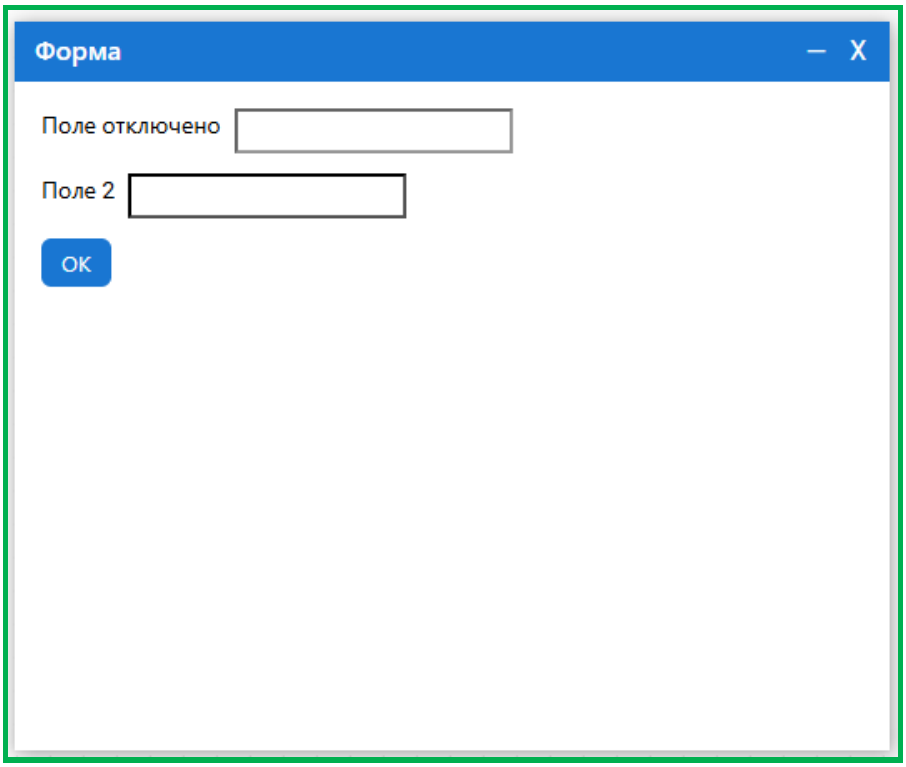
В данном случае выполняется отключение поля `field_1` и изменение заголовка данного поля на значение **Поле отключено**. Для этого в разъем **Значение по умолчанию** передается словарь, где ключ – ID элемента на форме, значения – словарь с изменяемыми параметрами данного элемента.

- `label` – заголовок поля (текстовое значение);
- `disabled` – признак, определяющий, недоступен ли элемент для взаимодействия (допустимое значение: Истина/Ложь).

Важно: Полный список доступных параметров можно найти в разделе **Доступные параметры для динамического изменения**.



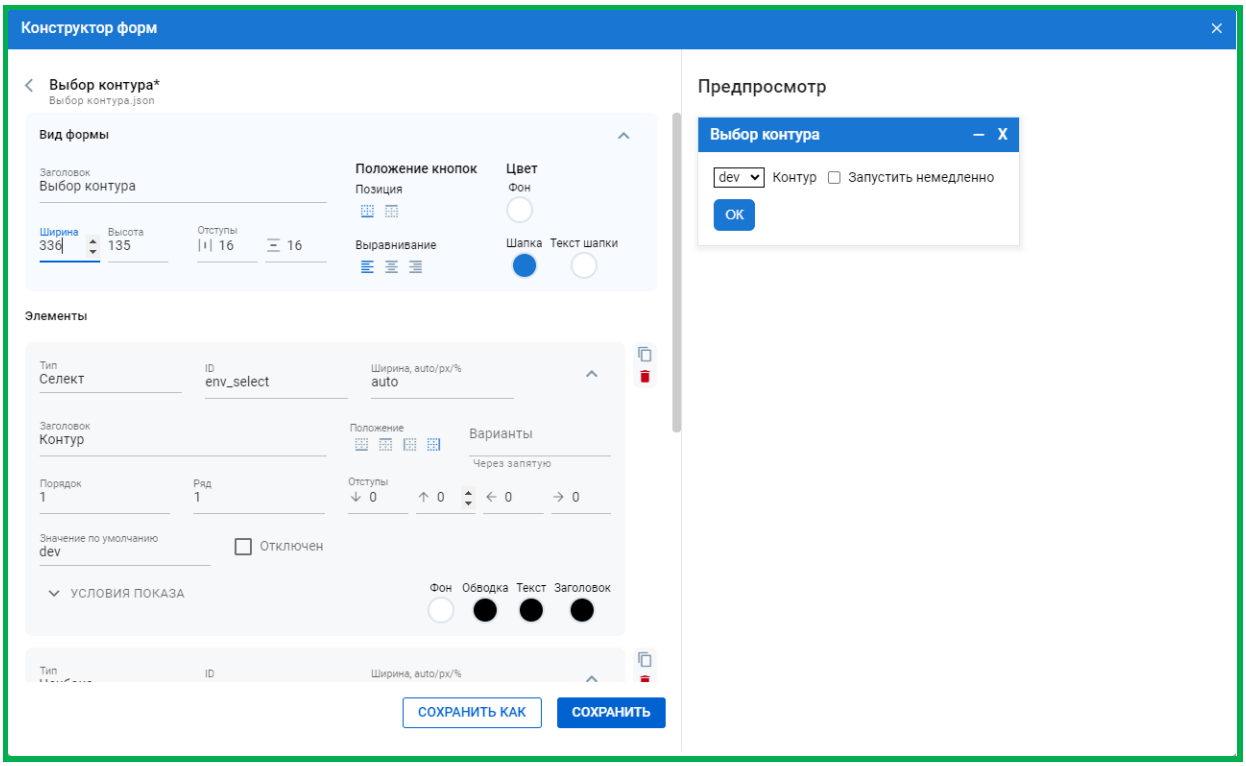
После запуска формы будет отключено поле с ID `field_1` и изменен заголовок.



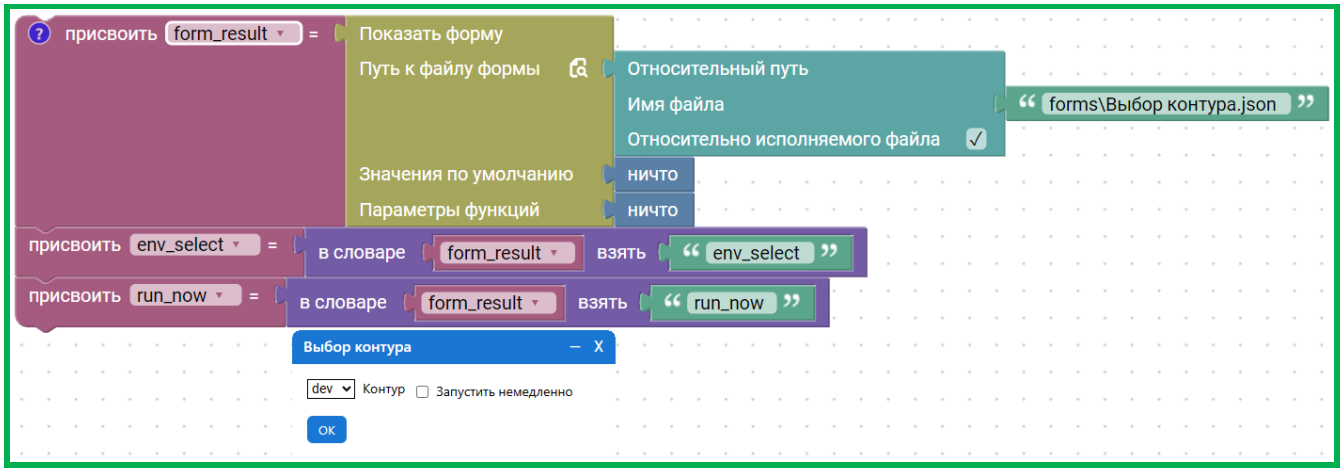
Пример 6 – Выбор окружения перед запуском робота

Робот запрашивает у пользователя окружение запуска и подтверждение перед стартом.

1. Создаётся форма с элементами: **Селект** (значения: dev, test, prod) и **Чекбокс** «Запустить немедленно»;



2. В алгоритме вызывается форма и из результата читаются ключи `env_select` и `run_now`;

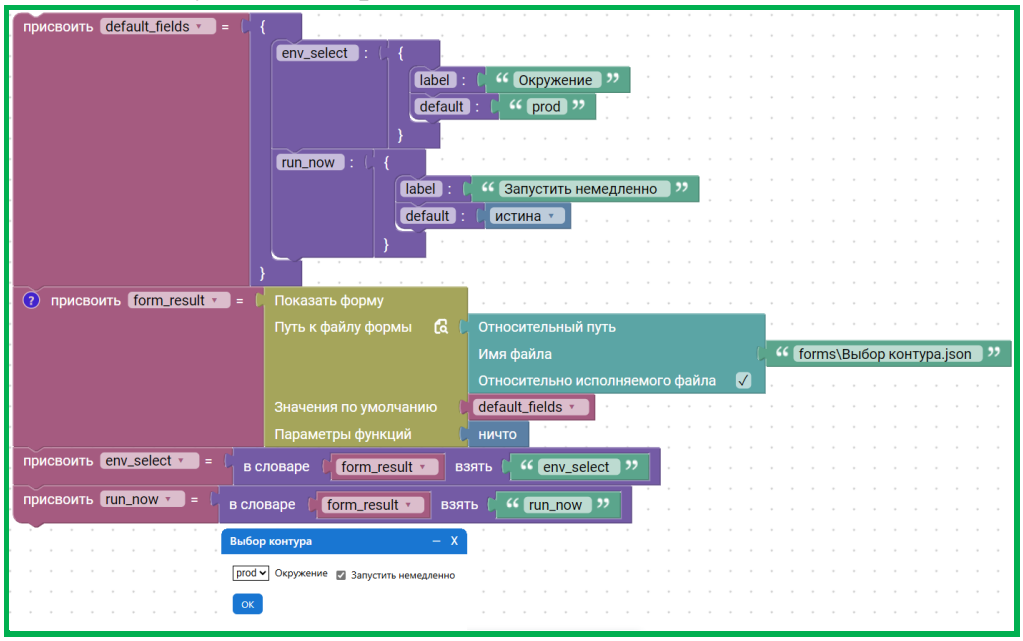


2. В зависимости от выбранного может изменяться логика выполнения алгоритма и выполняется запуск.

Пример словаря для динамического задания значений по умолчанию:

```
{
    'env_select': {
        'label': 'Окружение',
        'default': 'prod',
    },
    'run_now': {
        'label': 'Запустить немедленно',
        'default': True,
    }
}
```

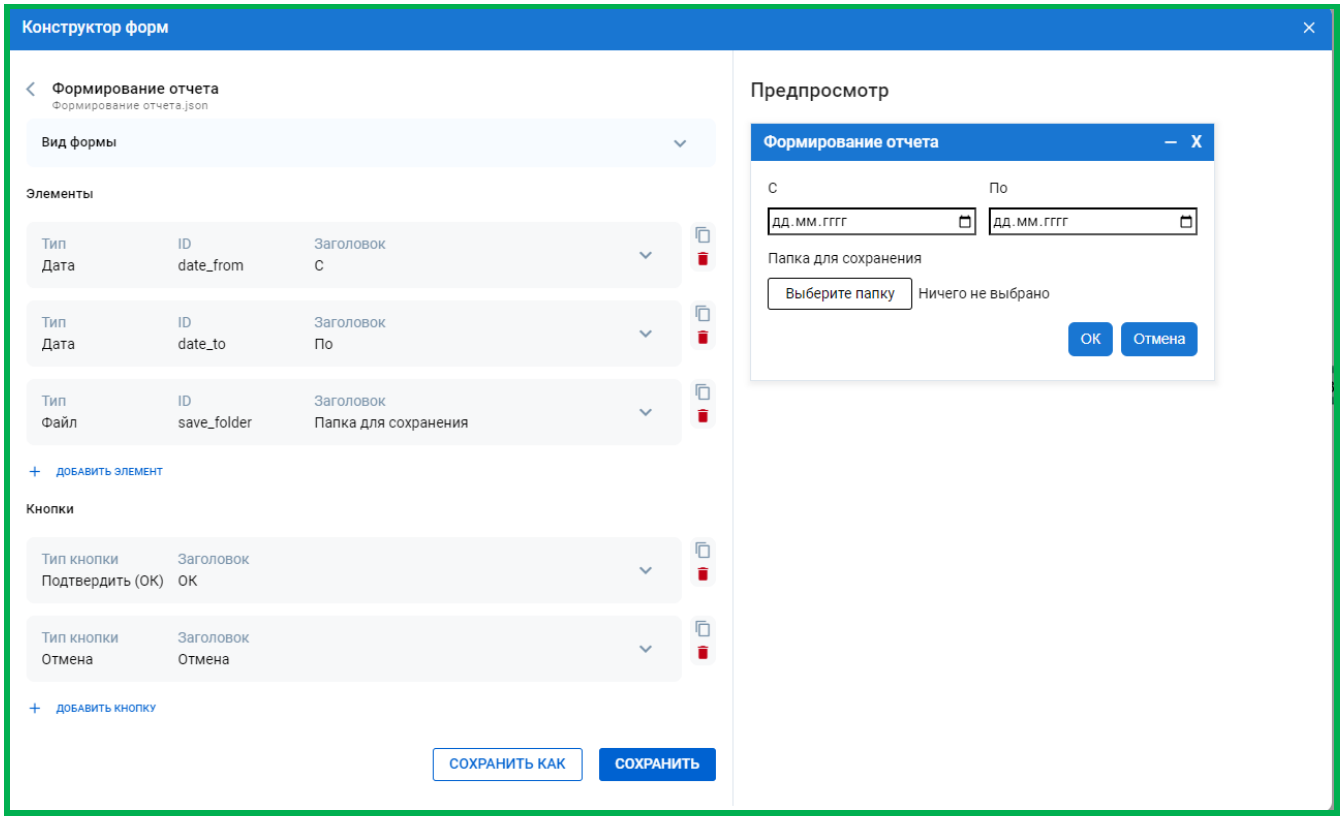
В случае задания динамических значений по умолчанию форма и алгоритм будут выглядеть следующим образом:



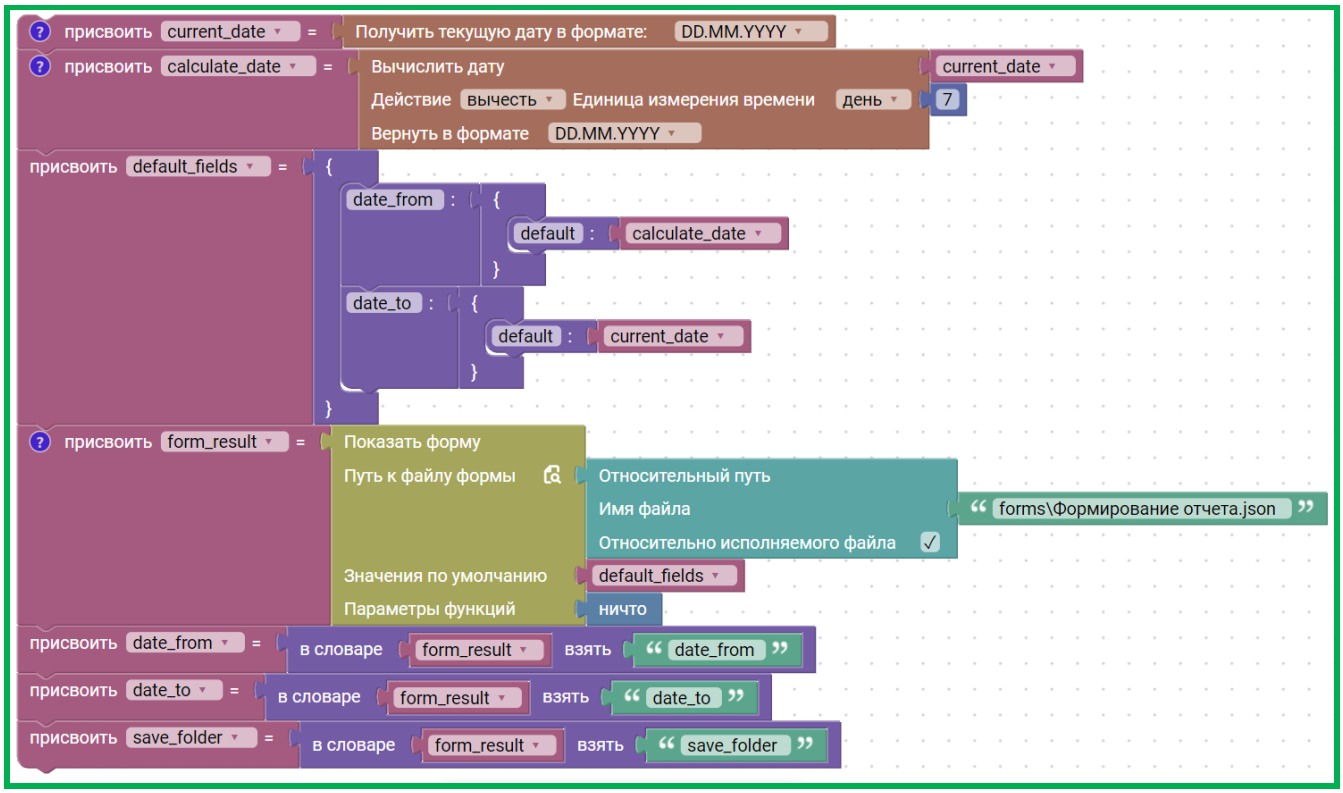
Пример 7 – Плановый запуск отчёта с параметрами периода

Робот перед запуском отчёта запрашивает у пользователя даты «с» и «по» и путь для сохранения.

1. Создаётся форма с полями: **Дата** (`date_from`), **Дата** (`date_to`) и **Файл** (выбор папки)



2. По умолчанию date_from = текущая дата минус 7 дней, date_to = текущая дата

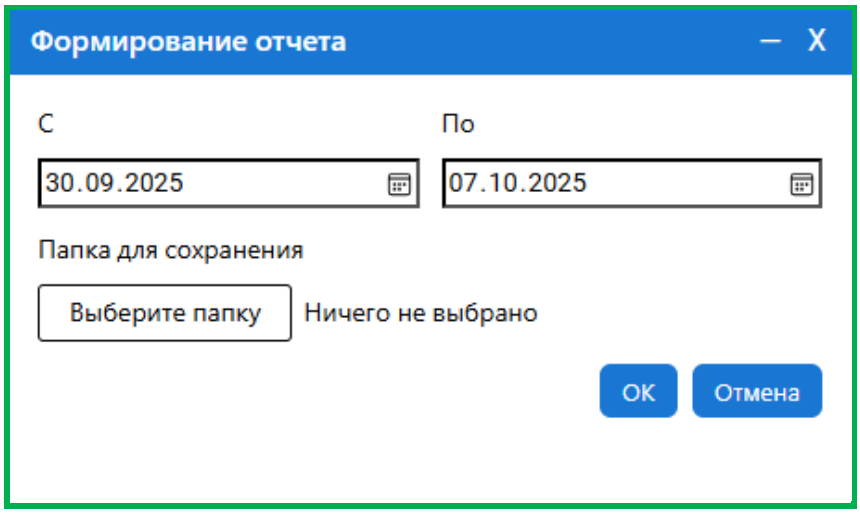


3. После подтверждения робот формирует отчёт за выбранный период и сохраняет в указанную директорию.

Пример словаря для установки дефолтных значений:

```
{
    'date_from': { 'default': '2025-10-01' },
    'date_to':   { 'default': '2025-10-08' }
}
```

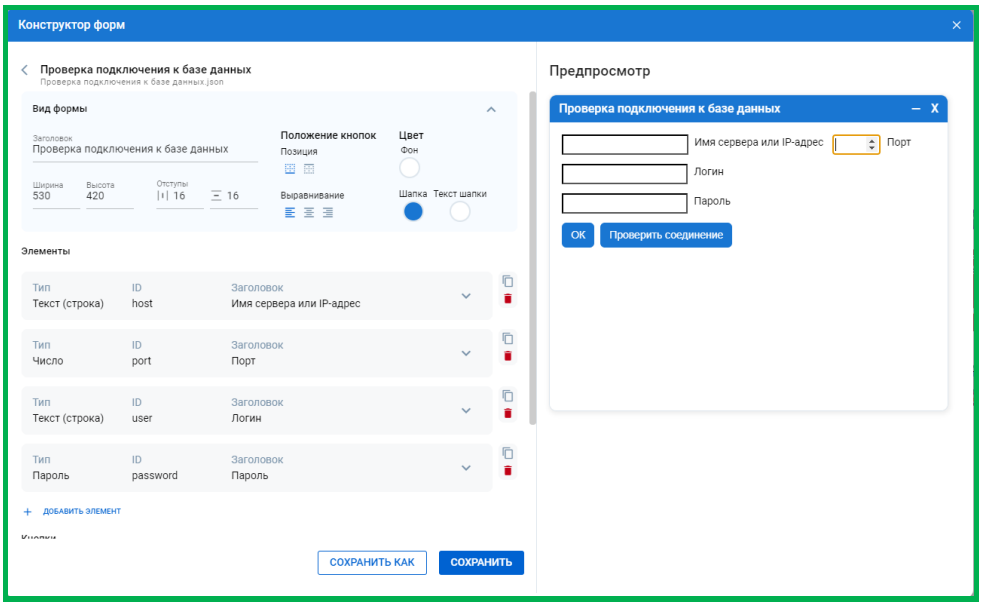
В случае задания динамических значений по умолчанию форма и алгоритм будут выглядеть следующим образом:



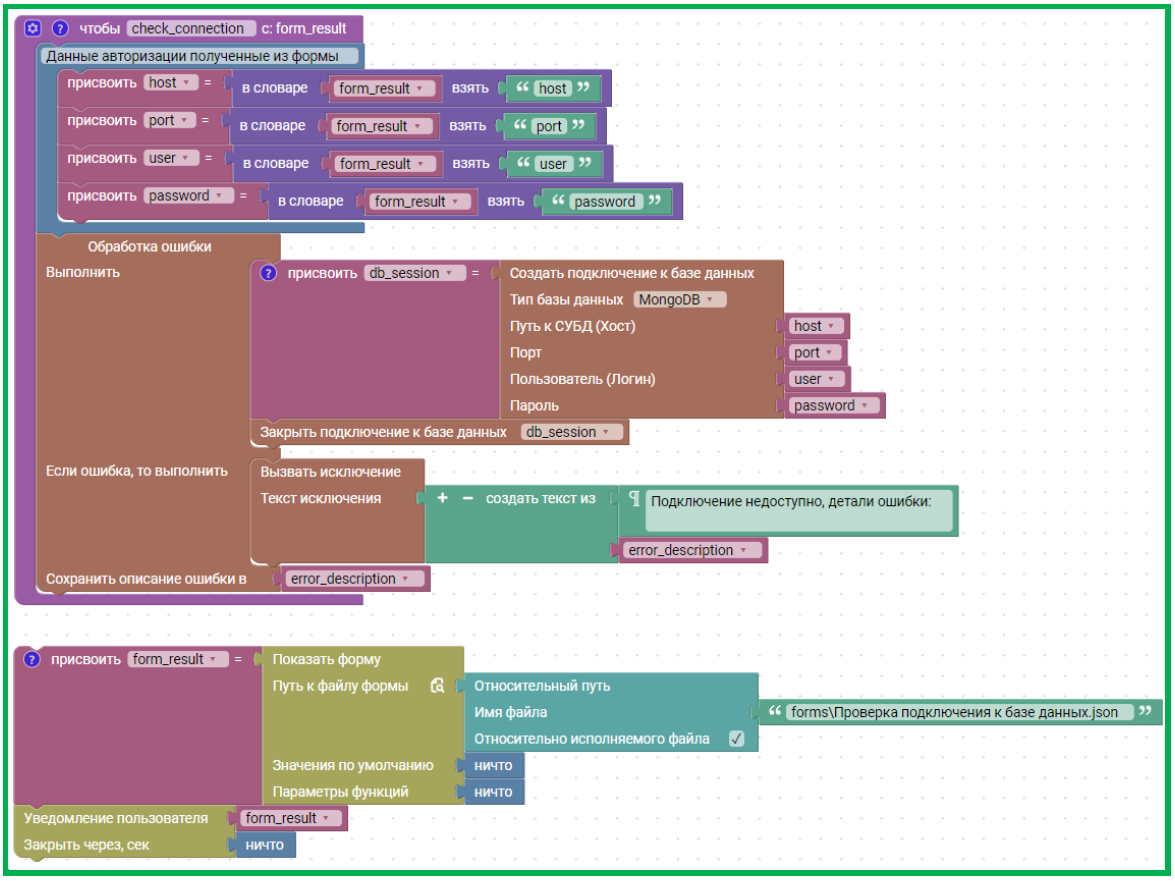
Пример 8 – Ввод параметров подключения к БД и проверка соединения

Перед выполнением записи в базу данных робот собирает параметры подключения и валидирует их.

1. Создаётся форма с полями: **Текст** `host`, **Число** `port`, **Текст** `user`, **Пароль** `password`;



2. Кнопка типа **Триггер** «Проверить соединение» вызывает функцию проекта **check_connection** для теста коннекта;



3. При успешной проверке пользователь подтверждает запуск основного сценария кнопкой «ОК».

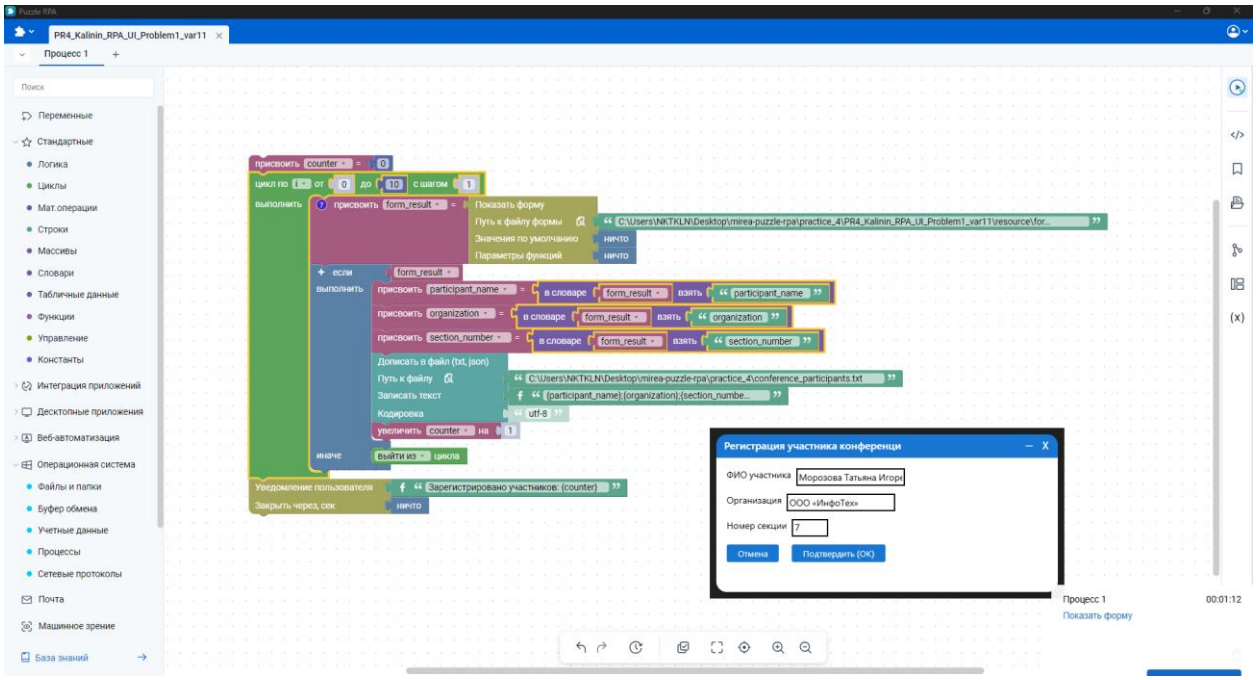
При успешной проверке подключения к базе данных, форма покажет статус выполнения функции **check_connection**.

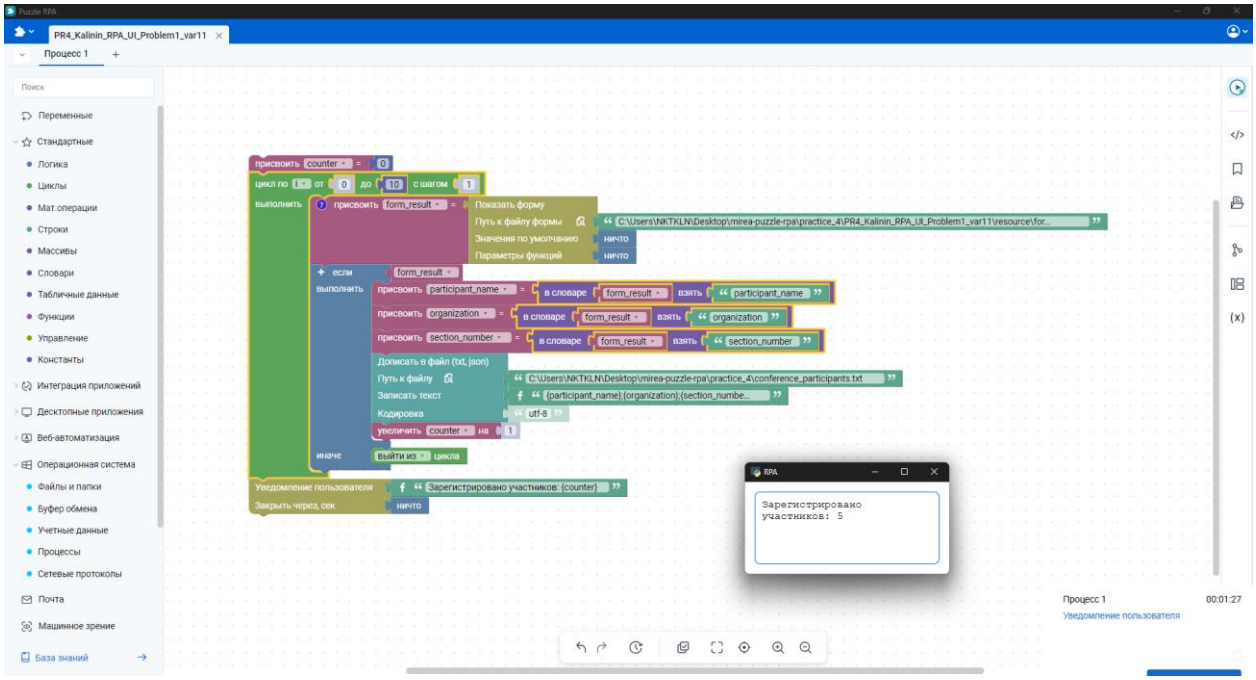
КОНТРОЛЬНЫЕ ЗАДАНИЯ

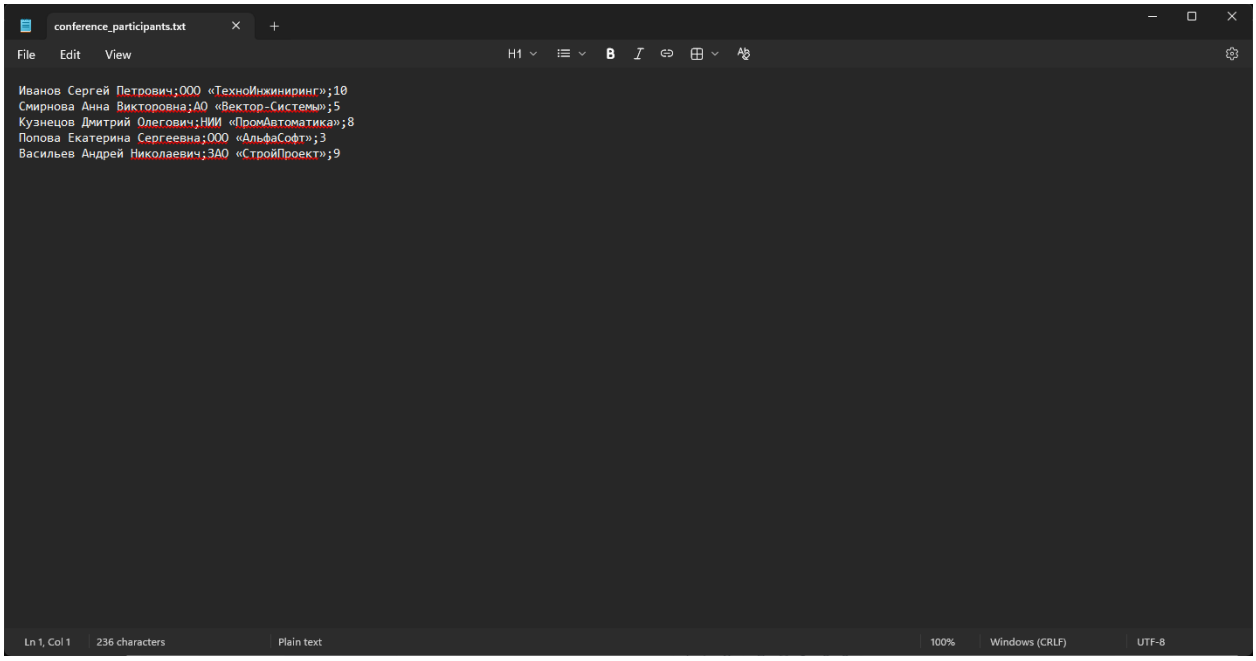
Контрольное задание 1

Условие задания варианта: Необходимо разработать систему регистрации участников конференции, которая с помощью пользовательской формы позволяет последовательно вводить данные докладчиков и слушателей, сохраняет информацию о каждом участнике в текстовый файл, поддерживает досрочное завершение ввода по нажатию кнопки отмены, а по окончании работы формирует итоговое сообщение с количеством зарегистрированных участников.

Успешное выполнение кода процесса





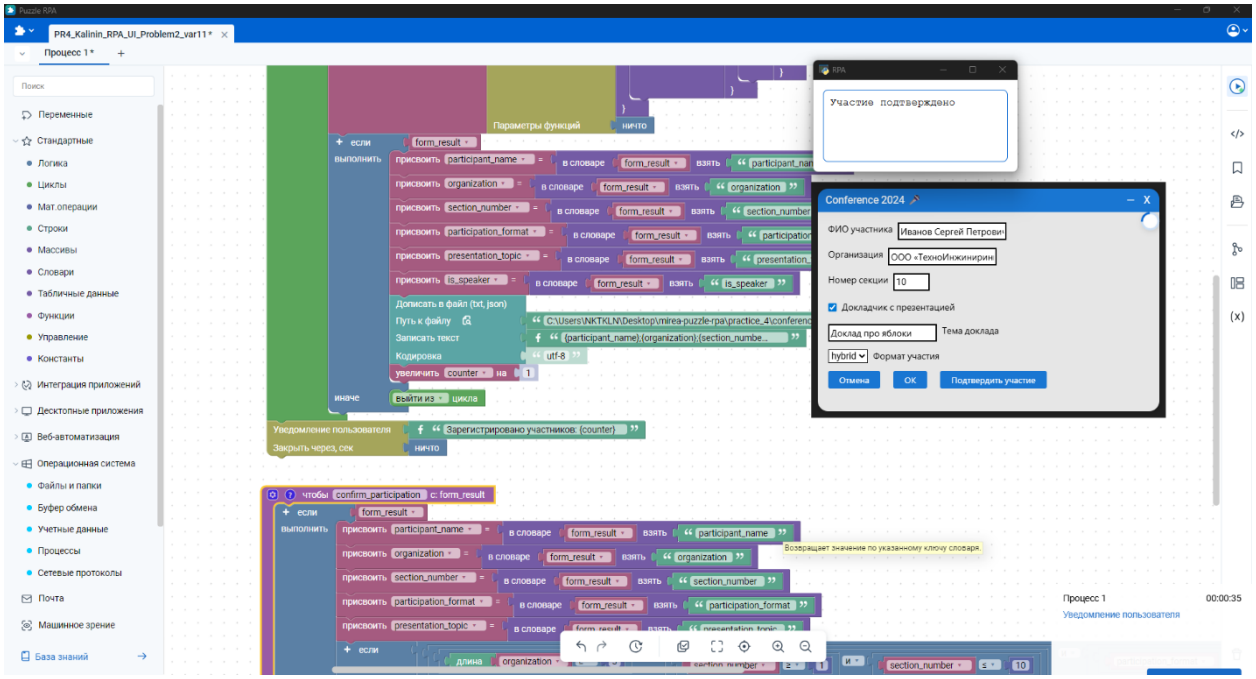


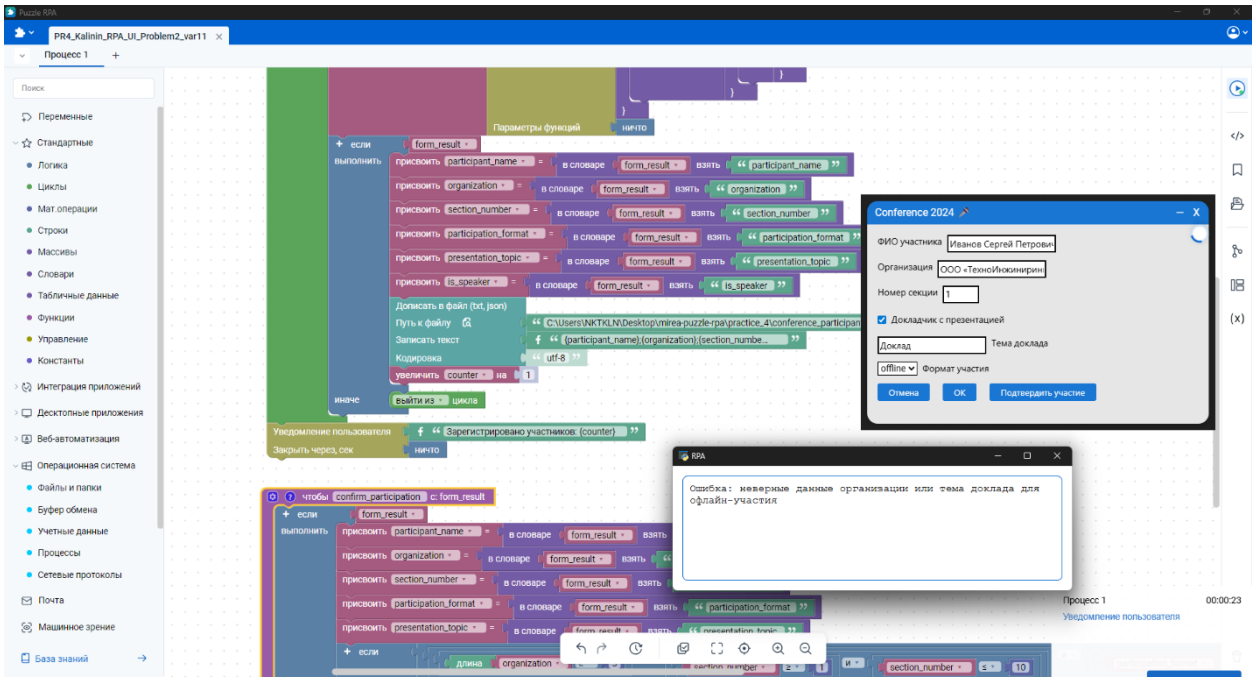
Алгоритм решения: организовать цикл вызова пользовательской формы регистрации участников, в котором на каждой итерации вводятся данные одного человека: ФИО, организация и номер секции. После подтверждения введённые данные проверяются и сохраняются в файл conference_participants.txt в формате ФИО;Организация;Номер_секции с новой строки. Если пользователь нажимает кнопку отмены, ввод данных досрочно прекращается. По завершении работы алгоритм подсчитывает количество успешно зарегистрированных участников и выводит итоговое сообщение с этим числом.

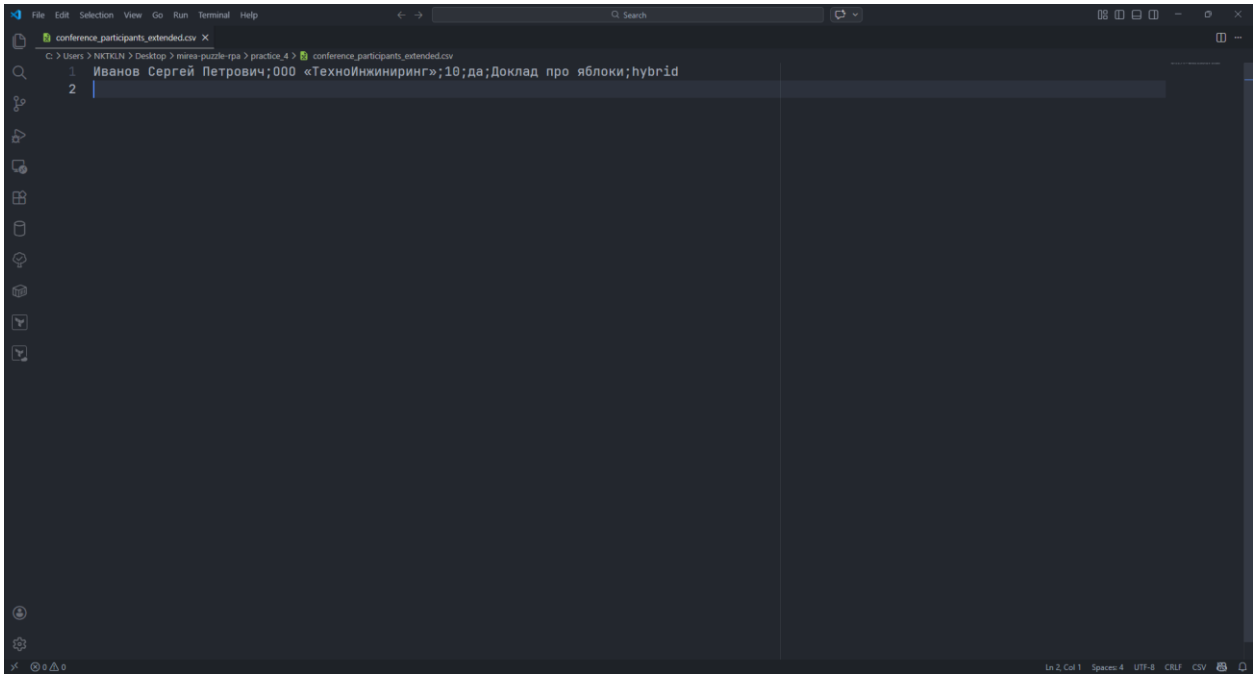
Контрольное задание 2

Условие задания варианта: необходимо разработать расширенную систему регистрации участников конференции, которая использует пользовательскую форму для последовательного ввода данных о каждом участнике, включая ФИО, организацию, номер секции, формат участия и дополнительные сведения о докладе. Форма должна поддерживать динамическое отображение поля темы доклада только для докладчиков с презентацией, использовать специальную кнопку-триггер для предварительной проверки корректности введённых данных без закрытия формы, а после успешного подтверждения сохранять информацию в файл conference_participants_extended.csv. По итогам работы система должна обеспечить поочерёдную регистрацию до 10 участников с возможностью досрочного завершения через кнопку отмены.

Успешное выполнение кода процесса







Алгоритм решения: организовать последовательный ввод данных 10 участников через форму «Расширенная регистрация конференции», заранее задав программные параметры формы: заголовок окна Conference 2024 🎤, цвет фона и значение поля participation_format = hybrid по умолчанию. На каждой итерации пользователь вводит данные участника, при этом поле темы доклада отображается только если отмечен чекбокс докладчика. Перед окончательным сохранением обязательно вызывается функция confirm_participation(form_result) через кнопку-триггер, которая проверяет корректность организации, номера секции и условий для офлайн-участия, после чего выводит сообщение об успешном подтверждении либо об ошибке. Если после проверки нажата кнопка ОК, данные сохраняются в файл conference_participants_extended.csv в формате ФИО;Организация;Номер_секции;Докладчик;Тема_доклада;Формат. Если пользователь нажимает Отмена и форма возвращает пустое значение, выполнение алгоритма досрочно прекращается.

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_4

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/ph65kkookict61hhn7